

claude-windows / 02b9e9c9-3806-45ca-9b24-2ecc35a81efd

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\02b9e9c9-3806-45ca-9b24-2ecc35a81efd\subagents\workflows\wf_27df53c8-8e0\agent-a5cacacfccba86c33.jsonl`  
- SHA-256: `87eac739693337fb7b6e48ca31e7dbbf3a8700df4ae1280f39c2d3eb21e62a6b`  
- Source modified: `2026-06-10T16:22:08+00:00`  
- Imported at: `2026-07-05T16:48:21+00:00`  
- project: `wf_27df53c8-8e0`  
- session_id: `02b9e9c9-3806-45ca-9b24-2ecc35a81efd`
```

Transcript

1. user (2026-06-10T16:13:27.749Z)

You are doing a DEEP code-correctness and crypto review of the core (non-parser) modules of muneem, a single-user, local-first personal-finance bookkeeper in Python. Repo:

C:/Users/avidu/Projects/muneem (src layout, package at src/muneem). It stores the user's entire financial life in a SQLCipher-encrypted SQLite ledger; the 256-bit key lives in the OS keychain (Windows is the primary platform); there is a passphrase-wrapped recovery file (Argon2id + AES-GCM).

SENSITIVE-DATA RULES (mandatory, highest priority):

- testdata/aubank-statements/, testdata/axis-statements/, testdata/hdfc-statements/, testdata/snapshots/, and testdata/credentials/ in the muneem repo contain the user's REAL financial documents and a real OAuth client file. Do NOT open/read the PDFs. You MAY check file existence and git-tracking status (git ls-files, git log --all -- <path>).
- If you must inspect testdata/snapshots/*.json or testdata/credentials/*.json to assess sensitivity, read at most the JSON key structure and NEVER quote values (no account numbers, balances, person names, client secrets, transaction amounts) in your output.
- Never include sensitive values in your findings. Refer to them abstractly ("a real account number", "a client_secret value").
- Do not use WebSearch/WebFetch and do not send any repo content to any external service.

Read EVERY one of these files completely: src/muneem/db.py, migrations.py, db_key.py, recovery.py, secrets.py, unlock.py, statement_unlock.py, password_rules.py, config.py, cli.py, errors.py, log.py, redaction.py, extract.py, ocr.py, offers.py. Also skim tests that cover them when you need to confirm intended behavior (tests/test_db*.py, test_migrations.py, test_recovery.py, test_secrets.py, test_unlock.py, test_statement_unlock.py).

Hunt specifically for:

- Crypto misuse: AES-GCM nonce handling, Argon2id parameters, key material lifetime in memory, key hex-encoding into PRAGMA strings (SQL injection via key? quoting?), silent plaintext fallback paths, rekey/backup atomicity and rollback correctness.
- SQLite/SQLCipher correctness: PRAGMA ordering (key before any access), WAL + backup API interaction, transaction boundaries, foreign-key enforcement gaps, migration runner edge cases (partial failure mid-migration, checksum verification timing, user_version vs schema_migrations divergence).
- Windows-specific bugs: path handling, file locking (replacing an open DB file), keyring backend behavior, console encoding.
- Error handling: swallowed exceptions, error paths that leave the DB/keychain/recovery file inconsistent, TOCTOU on file checks.
- CLI: argument handling, destructive operations without confirmation, sensitive values reaching stdout/argv/process lists.
- Logging/redaction: which modules log raw values; whether RedactingFilter patterns actually cover Indian account-number/PAN/card formats; logger adoption gaps.
- General Python correctness: mutable defaults, resource leaks, encoding assumptions, off-by-one, dead code.

Report ONLY real, defensible findings with exact file:line and quoted (non-sensitive) code

evidence. Severity honestly: critical = data loss / key compromise / silent corruption; high = wrong results or security weakness in realistic use. Do not pad with style nits ? but DO include medium/low items that are genuinely worth fixing. Your final output must be the structured findings.

2. user (2026-06-10T16:13:32.804Z)

```
1      """Local SQLite store: the ``documents`` and ``offers`` slice of the schema.
2
3      This is the minimal slice the promo spine needs (ROADMAP A.2). The full schema
4      (accounts, transactions, holdings, merchants, items) grows with later milestones.
5      ``documents.sha256`` is UNIQUE so re-ingesting the same file is a no-op.
6      ``offers.source`` records provenance ('text' = exact text layer, 'ocr' = approximate).
7      """
8
9      from __future__ import annotations
10
11      import os
12      import sqlite3
13      from collections.abc import Iterator
14      from contextlib import contextmanager
15      from pathlib import Path
16
17      from muneem.errors import EncryptedDatabaseError # re-exported so
db.EncryptedDatabaseError works
18
19      SCHEMA_VERSION = 4
20
21      # Unencrypted SQLite files start with this 16-byte magic header; an encrypted
(SQLCipher)
22      # file does not, so we can tell them apart without a key.
23      _SQLITE_MAGIC = b"SQLite format 3\x00"
24
25      _SCHEMA = """
26      CREATE TABLE IF NOT EXISTS documents (
27          id            INTEGER PRIMARY KEY,
28          source        TEXT NOT NULL,
29          external_id   TEXT,
30          sender        TEXT,
31          subject       TEXT,
32          received_date TEXT,
33          doc_type      TEXT,
34          sha256        TEXT NOT NULL UNIQUE,
35          path          TEXT,
36          status        TEXT NOT NULL DEFAULT 'parsed',
37          created_at    TEXT NOT NULL DEFAULT (datetime('now'))
38      );
39
40      CREATE TABLE IF NOT EXISTS offers (
41          id            INTEGER PRIMARY KEY,
42          document_id   INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
43          merchant      TEXT,
44          promo_code    TEXT,
45          description   TEXT,
46          expiry_raw    TEXT,
47          expiry_type   TEXT,
48          source        TEXT NOT NULL DEFAULT 'text',
49          redeemed      INTEGER NOT NULL DEFAULT 0,
50          source_text   TEXT
51      );
52
53      CREATE INDEX IF NOT EXISTS idx_offers_document ON offers(document_id);
54
55      -- We use separate tables for each bank (e.g. axis_transactions, hdfc_transactions)
56      -- instead of a unified table to preserve raw schema fidelity of the source PDFs.
```

```

57 -- Normalization into a unified view layer is planned for a later phase
58 -- once all bank schemas are stabilized.
59 CREATE TABLE IF NOT EXISTS axis_transactions (
60     id INTEGER PRIMARY KEY,
61     document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
62     account_number TEXT,
63     txn_date TEXT,
64     value_date TEXT,
65     particulars TEXT,
66     debit REAL,
67     credit REAL,
68     balance REAL
69 );
70
71 CREATE TABLE IF NOT EXISTS axis_cc_transactions (
72     id INTEGER PRIMARY KEY,
73     document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
74     account_number TEXT,
75     txn_date TEXT,
76     particulars TEXT,
77     merchant_category TEXT,
78     amount REAL,
79     type TEXT
80 );
81
82 CREATE TABLE IF NOT EXISTS au_transactions (
83     id INTEGER PRIMARY KEY,
84     document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
85     account_number TEXT,
86     txn_date TEXT,
87     value_date TEXT,
88     particulars TEXT,
89     debit REAL,
90     credit REAL,
91     balance REAL
92 );
93
94 CREATE TABLE IF NOT EXISTS hdfc_transactions (
95     id INTEGER PRIMARY KEY,
96     document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
97     account_number TEXT,
98     txn_date TEXT,
99     value_date TEXT,
100    particulars TEXT,
101    ref TEXT,
102    withdrawal REAL,
103    deposit REAL,
104    closing_balance REAL
105 );
106 """
107
108
109 def _raw_key_pragma(key: str) -> str:
110     """The SQLCipher raw-key PRAGMA: use the 256-bit ``key`` (64 hex chars) directly, no
KDF."""
111     return f"PRAGMA key = \"x'{key}'\""
112
113
114 def is_plaintext_sqlite(db_path: Path | str) -> bool:
115     """True iff the file exists and begins with the *unencrypted* SQLite magic header.
116
117     Lets us refuse to silently treat a plaintext ledger as encrypted (and vice-versa)
118     without needing the key.
119     """
120     try:

```

```

121         with open(db_path, "rb") as fh:
122             return fh.read(16) == _SQLITE_MAGIC
123     except OSError:
124         return False
125
126
127     def _harden_permissions(path: str) -> None:
128         """Best-effort: restrict the DB file to its owner (0600). No-op on failure or
129         ``:memory:``."""
129         if path == ":memory:":
130             return
131         try:
132             os.chmod(path, 0o600)
133         except OSError:
134             pass
135
136
137     def _open_encrypted(path: str, key: str) -> sqlite3.Connection:
138         """Open an encrypted connection via SQLCipher, verifying the key before returning.
139
140         A keyed open NEVER silently downgrades to plaintext: if the binding is missing, or
141         the
142         key is wrong (or the file is not an encrypted muneem ledger), this raises.
143         """
143         try:
144             import sqlcipher3
145         except ImportError as exc: # pragma: no cover - exercised only without the binding
146             raise EncryptedDatabaseError(
147                 "database encryption requested but the 'sqlcipher3' package is not
148                 installed"
149                 ) from exc
150
150         conn = sqlcipher3.connect(path)
151         conn.execute(_raw_key_pragma(key)) # MUST be the first statement on the connection
152         try:
153             conn.execute("SELECT count(*) FROM sqlite_master") # forces key verification
154         except sqlcipher3.DatabaseError as exc:
155             conn.close()
156             raise EncryptedDatabaseError(
157                 f"could not open encrypted database {Path(path).name}: wrong key, or not an
158                 "
159                 "encrypted muneem ledger"
160                 ) from exc
160         conn.row_factory = sqlcipher3.Row
161         return conn
162
163
164     @contextmanager
165     def connect(db_path: Path | str, *, key: str | None = None) ->
166     Iterator[sqlite3.Connection]:
166         """Open a connection (commit on success, always close). Parent dirs created.
167
168         With ``key`` (a 64-hex-char SQLCipher key) the database is **encrypted at rest** via
169         SQLCipher. Without it, a plaintext stdlib :mod:`sqlite3` connection is used ? the
170         path for
171         ``:memory:`` and tests. A keyed open never silently falls back to plaintext (see
172         :func:`_open_encrypted`); the production ledger is opened via
173         :func:`connect_encrypted`.
174         """
173         path = str(db_path)
174         if path != ":memory:":
175             Path(path).parent.mkdir(parents=True, exist_ok=True)
176         if key is not None:
177             conn = _open_encrypted(path, key)
178             _harden_permissions(path)

```

```

179         else:
180             conn = sqlite3.connect(path)
181             conn.row_factory = sqlite3.Row
182             conn.execute("PRAGMA foreign_keys = ON")
183             try:
184                 yield conn
185                 conn.commit()
186             finally:
187                 conn.close()
188
189
190 @contextmanager
191 def connect_encrypted(db_path: Path | str) -> Iterator[sqlite3.Connection]:
192     """Open the real on-disk ledger, encrypted with the keychain-managed key.
193
194     The production path: resolves (or mints, on first use) the 256-bit key from the OS
195     keychain and opens with SQLCipher. If ``db_path`` is a pre-existing *plaintext*
196     SQLite file, refuses with a clear pointer to ``muneem db encrypt`` rather than a cryptic
197     "file is not a database" error.
198     """
199     from muneem.db_key import get_or_create_key
200
201     path = str(db_path)
202     if is_plaintext_sqlite(path):
203         raise EncryptedDatabaseError(
204             f"{Path(path).name} is an unencrypted SQLite database; run 'muneem db
encrypt' "
205             "to migrate it to an encrypted ledger"
206         )
207     with connect(path, key=get_or_create_key()) as conn:
208         yield conn
209
210
211 def encrypt_database(db_path: Path | str, key: str) -> int:
212     """Migrate a plaintext SQLite ledger to an encrypted one in place, via
213     ``sqlcipher_export``.
214
215     The original is preserved as ``<name>.plaintext.bak`` for the caller to verify and
216     then securely delete (it still holds the cleartext data). Returns the number of user
217     tables copied. Raises :class:`EncryptedDatabaseError` if ``db_path`` is not a plaintext
218     SQLite file.
219     """
220     import sqlcipher3
221
222     path = Path(db_path)
223     if not is_plaintext_sqlite(path):
224         raise EncryptedDatabaseError(
225             f"{path.name} is not a plaintext SQLite database (already encrypted or
missing)"
226         )
227
228     enc_tmp = path.with_name(path.name + ".enc.tmp")
229     enc_tmp.unlink(missing_ok=True)
230     try:
231         src = sqlcipher3.connect(str(path)) # open the plaintext source (no key)
232         try:
233             src_tables = src.execute(
234                 "SELECT count(*) FROM sqlite_master WHERE type = 'table'"
235             ).fetchone()[0]
236             target = str(enc_tmp).replace("\\", "/") # SQL string literal: forward
slashes
237             src.execute(f"ATTACH DATABASE '{target}' AS encrypted KEY 'x'{key}'")

```

```

236         src.execute("SELECT sqlcipher_export('encrypted')")
237         src.execute("DETACH DATABASE encrypted")
238     finally:
239         src.close()
240
241     # Verify the encrypted copy BEFORE the destructive swap: it must open with the
key, pass
242     # an integrity check, and carry every table across. (The plaintext .bak is kept
either way,
243     # so a failed check is recoverable ? but we refuse to swap in a bad file.)
244     with connect(str(enc_tmp), key=key) as check:
245         integrity = check.execute("PRAGMA integrity_check").fetchone()[0]
246         ntables = check.execute(
247             "SELECT count(*) FROM sqlite_master WHERE type = 'table'"
248         ).fetchone()[0]
249     if integrity != "ok" or ntables != src_tables:
250         raise EncryptedDatabaseError(
251             f"encryption migration verification failed for {path.name} "
252             f"(integrity={integrity!r}, tables {ntables}/{src_tables}); original
left untouched"
253         )
254
255     backup = path.with_name(path.name + ".plaintext.bak")
256     backup.unlink(missing_ok=True)
257     path.rename(backup)
258     enc_tmp.rename(path) # consumed on success
259     _harden_permissions(str(path))
260     finally:
261         # Never leave a partial temp behind: no-op on success (already renamed), cleans
up on error.
262         enc_tmp.unlink(missing_ok=True)
263     return int(ntables)
264
265
266     def rekey_database(db_path: Path | str, current_key: str, new_key: str) -> None:
267         """Re-encrypt the ledger under ``new_key`` in place (SQLCipher ``PRAGMA rekey``).
268
269         Opens with ``current_key`` first (which verifies it), then rekeys. Raises
270         :class:`EncryptedDatabaseError` if ``current_key`` is wrong or the file is not an
encrypted
271         muneem ledger. The caller is responsible for updating the keychain entry to
``new_key`` only
272         after this returns successfully.
273         """
274         conn = _open_encrypted(str(db_path), current_key) # validates current_key
275         try:
276             conn.execute(f"PRAGMA rekey = '{new_key}'")
277             conn.commit()
278         finally:
279             conn.close()
280         _harden_permissions(str(db_path))
281
282
283     def backup_database(db_path: Path | str, dest: Path | str, key: str) -> None:
284         """Write an **encrypted** online-backup copy of the ledger to ``dest`` (same key).
285
286         Uses the SQLite Online Backup API for a consistent snapshot ? safe even with WAL
active, so no
287         naive file copy and no manual checkpoint needed. The destination is keyed with the
same key, so
288         it stays encrypted at rest. Raises :class:`EncryptedDatabaseError` if ``key`` is
wrong.
289         """
290         import sqlcipher3
291

```

```

292     dest_path = Path(dest)
293     dest_path.parent.mkdir(parents=True, exist_ok=True)
294     dest_path.unlink(missing_ok=True)
295     source = _open_encrypted(str(db_path), key) # validates key
296     try:
297         target = sqlcipher3.connect(str(dest_path))
298         try:
299             target.execute(_raw_key_pragma(key)) # key the destination so the backup is
encrypted
300             source.backup(target) # consistent snapshot of the live DB
301         finally:
302             target.close()
303     finally:
304         source.close()
305     _harden_permissions(str(dest_path))
306
307
308 def init_db(conn: sqlite3.Connection) -> None:
309     """Bring the schema up to date via the migration runner (creates tables on a fresh
DB).
310
311     The current ``_SCHEMA`` is the frozen *baseline* (version ``SCHEMA_VERSION``);
future changes
312     are forward migrations in :mod:`muneem.migrations`, recorded in
``schema_migrations``.
313     """
314     from muneem import migrations
315
316     migrations.run_migrations(conn, baseline_sql=_SCHEMA,
baseline_version=SCHEMA_VERSION)
317
318
319 def get_document_by_sha(conn: sqlite3.Connection, sha256: str) -> sqlite3.Row | None:
320     return conn.execute("SELECT * FROM documents WHERE sha256 = ?",
(sha256,)).fetchone()
321
322
323 def _insert_row(conn: sqlite3.Connection, table: str, fields: dict) -> int:
324     """Internal helper for the simple INSERT pattern used by documents, offers, and bank
txns."""
325     cols = ", ".join(fields)
326     placeholders = ", ".join("?" for _ in fields)
327     cur = conn.execute(
328         f"INSERT INTO {table} ({cols}) VALUES ({placeholders})",
329         tuple(fields.values()),
330     )
331     rowid = cur.lastrowid # always set after a successful INSERT; guard for the type-
checker
332     if rowid is None: # pragma: no cover - defensive; INSERT always sets lastrowid
333         raise RuntimeError(f"INSERT into {table} did not return a row id")
334     return int(rowid)
335
336
337 def insert_document(conn: sqlite3.Connection, **fields: object) -> int:
338     """Insert into documents table. Delegates to the shared helper for DRY."""
339     return _insert_row(conn, "documents", dict(fields))
340
341
342 def record_provenance(
343     conn: sqlite3.Connection,
344     *,
345     document_id: int,
346     source_type: str,
347     parser: str | None = None,
348     parser_version: str | None = None,

```

```

349         mapping_layer: str | None = None,
350         confidence: str | None = None,
351     ) -> None:
352         """Upsert the provenance row for a parse (one per document ? see DESIGN §8.7).
353
354         First-class but not per transaction row: every txn table FKs ``documents(id)`, so
a single
355         provenance record per document is reachable by join. Idempotent on ``document_id``
(a re-parse
356         overwrites rather than duplicating).
357         """
358         conn.execute(
359             "INSERT INTO provenance "
360             "(document_id, source_type, parser, parser_version, mapping_layer, confidence) "
361             "VALUES (?, ?, ?, ?, ?, ?) "
362             "ON CONFLICT(document_id) DO UPDATE SET "
363             "source_type=excluded.source_type, parser=excluded.parser, "
364             "parser_version=excluded.parser_version, mapping_layer=excluded.mapping_layer, "
365             "confidence=excluded.confidence",
366             (document_id, source_type, parser, parser_version, mapping_layer, confidence),
367         )
368
369
370     def record_parse(
371         conn: sqlite3.Connection,
372         document_id: int,
373         *,
374         confidence: str,
375         parser: str,
376         source_type: str = "pdf_statement",
377         parser_version: str | None = None,
378         mapping_layer: str = "deterministic",
379     ) -> None:
380         """Record a parse outcome: set ``documents.status`` and upsert provenance (one per
parse)."""
381         conn.execute("UPDATE documents SET status = ? WHERE id = ?", (confidence,
document_id))
382         record_provenance(
383             conn,
384             document_id=document_id,
385             source_type=source_type,
386             parser=parser,
387             parser_version=parser_version,
388             mapping_layer=mapping_layer,
389             confidence=confidence,
390         )
391
392
393     def get_provenance(conn: sqlite3.Connection, document_id: int) -> sqlite3.Row | None:
394         return conn.execute("SELECT * FROM provenance WHERE document_id = ?",
(document_id,)).fetchone()
395
396
397     def insert_offer(conn: sqlite3.Connection, **fields: object) -> int:
398         return _insert_row(conn, "offers", fields)
399
400
401     def list_offers(conn: sqlite3.Connection) -> list[sqlite3.Row]:
402         return conn.execute(
403             """
404             SELECT o.id, o.merchant, o.promo_code, o.expiry_raw, o.description, o.source,
405                    d.external_id, d.path
406             FROM offers o
407             JOIN documents d ON d.id = o.document_id
408             ORDER BY o.id

```



```

409         """
410     ).fetchall()
411
412
413     def insert_axis_transaction(conn: sqlite3.Connection, **fields: object) -> int:
414         return _insert_row(conn, "axis_transactions", fields)
415
416
417     def insert_axis_cc_transaction(conn: sqlite3.Connection, **fields: object) -> int:
418         return _insert_row(conn, "axis_cc_transactions", fields)
419
420
421     def list_axis_cc_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
422         return conn.execute(
423             "SELECT * FROM axis_cc_transactions ORDER BY id DESC LIMIT ?", (limit,)
424         ).fetchall()
425
426
427     def list_axis_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
428         return conn.execute(
429             "SELECT * FROM axis_transactions ORDER BY id DESC LIMIT ?", (limit,)
430         ).fetchall()
431
432
433     def insert_hdfc_transaction(conn: sqlite3.Connection, **fields: object) -> int:
434         return _insert_row(conn, "hdfc_transactions", fields)
435
436
437     def list_hdfc_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
438         return conn.execute(
439             "SELECT * FROM hdfc_transactions ORDER BY id DESC LIMIT ?", (limit,)
440         ).fetchall()
441
442
443     def insert_au_transaction(conn: sqlite3.Connection, **fields: object) -> int:
444         return _insert_row(conn, "au_transactions", fields)
445
446
447     def list_au_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
448         return conn.execute(
449             "SELECT * FROM au_transactions ORDER BY id DESC LIMIT ?", (limit,)
450         ).fetchall()
451

```

3. user (2026-06-10T16:13:33.274Z)

```

1     """Hand-rolled, versioned, checksummed schema migrations (feedbackJune2026 P6).
2
3     Deliberately not Alembic ? muneem stays raw-`sqlite3` / stdlib-light. Migrations are
an ordered
4     list; each is applied exactly once, inside the (possibly encrypted) connection, and
recorded in a
5     ``schema_migrations`` table with a checksum. The schema-lock test
(`tests/test_db_invariants.py`)
6     pins the resulting shape, so accidental drift and deliberate changes both surface in
review.
7
8     Model:
9     - Baseline. The current schema (`db._SCHEMA`, version `db.SCHEMA_VERSION`) is the
*frozen*
10     baseline ? never edit it for new changes; add a migration instead. A fresh DB gets the

```

```

baseline
11     applied; a legacy DB created before this framework (tables present, no
`scheme_migrations`) is
12     detected and its baseline row back-filled (no DDL re-run).
13     - **Forward migrations.** Entries in `MIGRATIONS` with ``version > baseline`` run in
order. (None
14         yet ? the first lands with the unified-schema work, feedbackJune2026 P3.)
15
16     Known limitation (single-user, reviewed migrations): a crash *between* applying a
migration's DDL
17     and recording it could require manual cleanup; per-migration transactions can be added
when the
18     first non-trivial migration lands.
19     """
20
21     from __future__ import annotations
22
23     import hashlib
24     import sqlite3
25     from dataclasses import dataclass
26
27     from muneem.errors import MigrationError
28
29     _SCHEMA_MIGRATIONS = """
30     CREATE TABLE IF NOT EXISTS schema_migrations (
31         version    INTEGER PRIMARY KEY,
32         name       TEXT NOT NULL,
33         checksum   TEXT NOT NULL,
34         applied_at TEXT NOT NULL DEFAULT (datetime('now'))
35     );
36     """
37
38
39     @dataclass(frozen=True)
40     class Migration:
41         """One forward migration: a version, a name, and the DDL to apply (run via
executescript)."""
42
43         version: int
44         name: str
45         sql: str
46
47         @property
48         def checksum(self) -> str:
49             return hashlib.sha256(self.sql.encode("utf-8")).hexdigest()
50
51
52     # --- Forward migrations (version > baseline), applied in ascending order
-----
53
54     # v5: provenance/confidence as a FIRST-CLASS but SEPARATE table (DESIGN §8.7,
feedbackJune2026 P3).
55     # Keyed by document_id (1:1) so every txn row reaches its provenance by the existing FK
join ? no
56     # per-row redundancy. source_type readies it for non-PDF sources (aa/sms) we may add
later.
57     _M5_PROVENANCE = """
58     CREATE TABLE IF NOT EXISTS provenance (
59         id            INTEGER PRIMARY KEY,
60         document_id   INTEGER NOT NULL UNIQUE REFERENCES documents(id) ON DELETE CASCADE,
61         source_type   TEXT NOT NULL,
62         parser        TEXT,
63         parser_version TEXT,
64         mapping_layer TEXT,
65         confidence    TEXT,

```

```

66         created_at      TEXT NOT NULL DEFAULT (datetime('now'))
67     );
68     """
69
70     MIGRATIONS: list[Migration] = [
71         Migration(version=5, name="provenance", sql=_M5_PROVENANCE),
72     ]
73
74
75     def _checksum(sql: str) -> str:
76         return hashlib.sha256(sql.encode("utf-8")).hexdigest()
77
78
79     def _user_tables(conn: sqlite3.Connection) -> list[str]:
80         return [
81             r[0]
82             for r in conn.execute(
83                 "SELECT name FROM sqlite_master WHERE type='table' "
84                 "AND name NOT LIKE 'sqlite_%' AND name != 'schema_migrations'"
85             )
86         ]
87
88
89     def _record(conn: sqlite3.Connection, version: int, name: str, checksum: str) -> None:
90         conn.execute(
91             "INSERT INTO schema_migrations (version, name, checksum) VALUES (?, ?, ?)",
92             (version, name, checksum),
93         )
94
95
96     def run_migrations(
97         conn: sqlite3.Connection,
98         *,
99         baseline_sql: str,
100         baseline_version: int,
101         migrations: list[Migration] | None = None,
102     ) -> int:
103         """Bring ``conn``'s schema up to date. Returns the resulting ``user_version``.
104
105         Idempotent: re-running applies nothing already recorded. Raises
106         :class:`MigrationError` if the
107         frozen baseline's checksum no longer matches what was recorded (i.e. someone edited
108         the baseline
109         instead of adding a migration).
110         """
111         migrations = MIGRATIONS if migrations is None else migrations
112         conn.executescript(_SCHEMA_MIGRATIONS) # bookkeeping table first (idempotent)
113         applied = {r[0] for r in conn.execute("SELECT version FROM schema_migrations")}
114         current = conn.execute("PRAGMA user_version").fetchone()[0]
115         baseline_checksum = _checksum(baseline_sql)
116
117         if baseline_version in applied:
118             recorded = conn.execute(
119                 "SELECT checksum FROM schema_migrations WHERE version = ?",
120                 (baseline_version,)
121             ).fetchone()
122             if recorded and recorded[0] != baseline_checksum:
123                 raise MigrationError(
124                     "baseline schema changed without a migration ? add a migration instead
125                     of editing "
126                     "the frozen baseline (feedbackJune2026 P6)"
127                 )
128         else:
129             # Fresh DB ? create the baseline; legacy DB (tables already present) ? just
130             record it.

```

```

126         if not _user_tables(conn):
127             conn.executescript(baseline_sql)
128         _record(conn, baseline_version, "baseline", baseline_checksum)
129         if current < baseline_version:
130             conn.execute(f"PRAGMA user_version = {baseline_version}")
131             current = baseline_version
132
133     for migration in sorted(migrations, key=lambda m: m.version):
134         if migration.version <= baseline_version or migration.version in applied:
135             continue
136         conn.executescript(migration.sql)
137         _record(conn, migration.version, migration.name, migration.checksum)
138         conn.execute(f"PRAGMA user_version = {migration.version}")
139         current = migration.version
140
141     return int(current)
142

```

4. user (2026-06-10T16:13:33.744Z)

```

1     """The ledger's encryption key ? minted once, kept in the OS keychain.
2
3     muneem's local ledger is **encrypted at rest** with SQLCipher (DESIGN.md §5.4 rule 4).
4     The 256-bit key is generated on first use and stored in the OS keychain via
5     :mod:`muneem.secrets` (Windows Credential Manager / macOS Keychain / Secret Service) ?
6     never on disk in plaintext, never in the repo, never logged.
7
8     It is handed to SQLCipher in **raw-key form** (`PRAGMA key = "x'<64 hex>'"), which
uses
9     the 256-bit key directly and *skips* the passphrase KDF. That is correct here because
the
10    key is already full-entropy random; KDF stretching only buys security for low-entropy
human
11    passphrases (an opt-in we may add later ? see ROADMAP D.2 phased plan).
12    """
13
14    from __future__ import annotations
15
16    import secrets as _stdlib_secrets
17
18    from muneem import secrets as _keychain
19
20    # Keychain entry name (under the shared "muneem" service) holding the 64-hex-char DB
key.
21    _DB_KEY_NAME = "db-key"
22
23
24    def get_or_create_key() -> str:
25        """Return the ledger's 256-bit key as 64 hex chars, minting+storing one on first
use."""
26        existing = _keychain.get_secret(_DB_KEY_NAME)
27        if existing:
28            return existing
29        key = _stdlib_secrets.token_bytes(32).hex()
30        _keychain.set_secret(_DB_KEY_NAME, key)
31        return key
32
33
34    def peek_key() -> str | None:
35        """Return the stored key, or `None` if none has been minted yet. Never creates
one."""
36        return _keychain.get_secret(_DB_KEY_NAME)
37
38
39    def new_key() -> str:

```

```

40         """Generate a fresh random 256-bit key as 64 hex chars (not stored)."""
41         return _stdlib_secrets.token_bytes(32).hex()
42
43
44     def set_key(key: str) -> None:
45         """Overwrite the stored DB key ? used by key rotation, **after** the file has been
rekeyed."""
46         _keychain.set_secret(_DB_KEY_NAME, key)
47

```

5. user (2026-06-10T16:13:34.118Z)

```

1         """Disaster recovery for the encrypted ledger ? a passphrase-wrapped recovery file.
2
3         The DB key lives in the OS keychain (fast, full-entropy). If that keychain is ever lost
? OS
4         reinstall, new machine, profile/Credential-Manager corruption ? the ledger **and every
same-key
5         backup** become permanently unrecoverable. This module lets the user wrap that key under
a
6         passphrase they choose, into a recovery file they back up **off-machine**; with the file
*and* the
7         passphrase
8         they can restore the key into a fresh keychain.
9
10        Crypto: **Argon2id** (memory-hard KDF) derives a 256-bit wrapping key from the
passphrase + a random
11        salt; **AES-256-GCM** (authenticated) wraps the DB key. The recovery file holds only
12        ``{version, kdf params, salt, nonce, ciphertext}`` ? never the key, never the
passphrase. A wrong
13        passphrase fails the GCM authentication tag ? a clean error, with no decryption oracle.
14        """
15
16        from __future__ import annotations
17
18        import base64
19        import json
20        import os
21        from pathlib import Path
22        from typing import Any
23
24        from cryptography.exceptions import InvalidTag
25        from cryptography.hazmat.primitives.ciphers.aead import AESGCM
26        from cryptography.hazmat.primitives.kdf.argon2 import Argon2id
27
28        from muneem.errors import MuneemError
29
30        RECOVERY_VERSION = 1
31        _AAD = b"muneem-recovery-v1" # binds the ciphertext to this format/version
32
33        # Argon2id parameters (memory in KiB). Conservative-but-real; the file stores them so
they can be
34        # raised over time without breaking older recovery files.
35        _TIME_COST = 3
36        _MEMORY_KIB = 64 * 1024 # 64 MiB
37        _LANES = 4
38        _SALT_BYTES = 16
39        _NONCE_BYTES = 12
40        _KEY_BYTES = 32
41        _MIN_PASSPHRASE = 8
42
43
44        class RecoveryError(MuneemError):
45            """A recovery file could not be created, or could not be opened (wrong passphrase /
corrupt)."""

```

```

46
47
48     def _derive(passphrase: str, salt: bytes, *, time_cost: int, memory_kib: int, lanes:
int) -> bytes:
49         kdf = Argon2id(
50             salt=salt,
51             length=_KEY_BYTES,
52             iterations=time_cost,
53             lanes=lanes,
54             memory_cost=memory_kib,
55         )
56         return kdf.derive(passphrase.encode("utf-8"))
57
58
59     def wrap_key(db_key: str, passphrase: str) -> dict[str, Any]:
60         """Return a JSON-serialisable recovery payload wrapping ``db_key`` under
61         ``passphrase``."""
62         if not passphrase or len(passphrase) < _MIN_PASSPHRASE:
63             raise RecoveryError(f"recovery passphrase must be at least {_MIN_PASSPHRASE}
characters")
64         salt = os.urandom(_SALT_BYTES)
65         nonce = os.urandom(_NONCE_BYTES)
66         key = _derive(passphrase, salt, time_cost=_TIME_COST, memory_kib=_MEMORY_KIB,
lanes=_LANES)
67         ciphertext = AESGCM(key).encrypt(nonce, db_key.encode("utf-8"), _AAD)
68         return {
69             "version": RECOVERY_VERSION,
70             "kdf": {
71                 "algo": "argon2id",
72                 "time_cost": _TIME_COST,
73                 "memory_kib": _MEMORY_KIB,
74                 "lanes": _LANES,
75             },
76             "salt": base64.b64encode(salt).decode(),
77             "nonce": base64.b64encode(nonce).decode(),
78             "ciphertext": base64.b64encode(ciphertext).decode(),
79         }
80
81     def unwrap_key(payload: dict[str, Any], passphrase: str) -> str:
82         """Recover the DB key from a recovery ``payload`` + ``passphrase``. Raises
RecoveryError."""
83         try:
84             salt = base64.b64decode(payload["salt"])
85             nonce = base64.b64decode(payload["nonce"])
86             ciphertext = base64.b64decode(payload["ciphertext"])
87             kdf = payload.get("kdf", {})
88         except (KeyError, ValueError, TypeError) as exc:
89             raise RecoveryError("recovery file is malformed") from exc
90         key = _derive(
91             passphrase,
92             salt,
93             time_cost=int(kdf.get("time_cost", _TIME_COST)),
94             memory_kib=int(kdf.get("memory_kib", _MEMORY_KIB)),
95             lanes=int(kdf.get("lanes", _LANES)),
96         )
97         try:
98             plain = AESGCM(key).decrypt(nonce, ciphertext, _AAD)
99         except InvalidTag as exc:
100             raise RecoveryError("wrong passphrase or corrupt recovery file") from exc
101         return plain.decode("utf-8")
102
103
104     def write_recovery_file(path: Path | str, db_key: str, passphrase: str) -> Path:
105         """Wrap ``db_key`` under ``passphrase`` and write the recovery file (owner-only

```

```

perms)."""
106     payload = wrap_key(db_key, passphrase)
107     out = Path(path)
108     out.parent.mkdir(parents=True, exist_ok=True)
109     out.write_text(json.dumps(payload, indent=2), encoding="utf-8")
110     try:
111         os.chmod(out, 0o600)
112     except OSError: # pragma: no cover - best-effort on platforms without POSIX perms
113         pass
114     return out
115
116
117     def read_recovery_file(path: Path | str, passphrase: str) -> str:
118         """Read a recovery file and return the unwrapped DB key. Raises RecoveryError on any
119 failure."""
119         p = Path(path)
120         try:
121             payload = json.loads(p.read_text(encoding="utf-8"))
122             except (OSError, ValueError) as exc:
123                 raise RecoveryError(f"could not read recovery file {p.name}") from exc
124             return unwrap_key(payload, passphrase)
125

```

6. user (2026-06-10T16:13:39.426Z)

```

1     """Secret storage via the OS keychain.
2
3     OAuth tokens, cached statement passwords, and the DB encryption key are stored here
4 (Windows
4     Credential Manager / macOS Keychain / Secret Service), never in the repo, on disk in
5 plaintext, or
5     in logs. See DESIGN.md section 5.
6
7     External-call discipline (every keyring call is wrapped with attribution):
8 - **read** (:func:`get_secret`) is *graceful* ? a missing or unavailable backend returns
9 ``None``
9     (absence is a valid state; this also keeps headless CI happy).
10 - **write** (:func:`set_secret`) is *loud* ? a failure raises
11 :class:`~muneem.errors.KeychainError`,
11     because silently losing a write (e.g. the DB key) would be dangerous. The secret
12 *name* is
12     included for diagnosis; the value never is.
13     """
14
15     from __future__ import annotations
16
17     import keyring
18     from keyring.errors import KeyringError, PasswordDeleteError
19
20     from muneem.errors import KeychainError
21
22     _SERVICE = "muneem"
23
24
25     def get_secret(name: str) -> str | None:
26         """Return the stored secret, or ``None`` if absent or the keychain is
27 unavailable."""
27         try:
28             return keyring.get_password(_SERVICE, name)
29         except KeyringError:
30             return None
31
32
33     def set_secret(name: str, value: str) -> None:
34         """Store (or replace) a secret in the OS keychain. Raises KeychainError on

```

```

failure."""
35         try:
36             keyring.set_password(_SERVICE, name, value)
37         except KeyringError as exc:
38             # Name only ? never the value ? in the message (it may be a password or the DB
key).
39             raise KeychainError(f"could not store secret {name!r} in the OS keychain") from
exc
40
41
42     def delete_secret(name: str) -> None:
43         """Delete a secret if present; a no-op if it is not (or the keychain is
unavailable)."""
44         try:
45             keyring.delete_password(_SERVICE, name)
46         except (PasswordDeleteError, KeyringError):
47             pass
48

```

7. user (2026-06-10T16:13:39.893Z)

```

1     """Decrypt password-protected statement PDFs (Stage 2).
2
3     Uses pypdf (BSD-3-Clause) ? permissive, pure-Python. Implements the hybrid
4     password strategy from DESIGN.md § 8.2: try a candidate dictionary first, then
5     fall back to an interactive prompt. The working password is returned so the
6     caller can cache it in the OS keychain (muneem.secrets); **it is never logged**.
7     """
8
9     from __future__ import annotations
10
11     import shutil
12     from collections.abc import Callable, Iterable
13     from dataclasses import dataclass
14     from pathlib import Path
15
16     import pypdf
17
18
19     @dataclass
20     class UnlockResult:
21         ok: bool
22         method: str | None # "plaintext" | "candidate" | "prompt"
23         password: str | None # working password ? do NOT log; None if plaintext/failed
24         output_path: Path | None # decrypted copy, if a destination was given
25
26
27     def is_encrypted(path: Path | str) -> bool:
28         return pypdf.PdfReader(str(path)).is_encrypted
29
30
31     def password_works(path: Path | str, password: str) -> bool:
32         reader = pypdf.PdfReader(str(path))
33         if not reader.is_encrypted:
34             return True
35         try:
36             return reader.decrypt(password) != pypdf.PasswordType.NOT_DECRYPTED
37         except Exception:
38             return False
39
40
41     def write_decrypted(path: Path | str, password: str | None, dest: Path | str) -> Path:
42         """Write a decrypted copy of the PDF to dest (password may be '' for plaintext)."""
43         src = Path(path)
44         dest = Path(dest)

```



```

45     dest.parent.mkdir(parents=True, exist_ok=True)
46     reader = pypdf.PdfReader(str(src))
47     if reader.is_encrypted:
48         reader.decrypt(password or "")
49     writer = pypdf.PdfWriter()
50     writer.append(reader)
51     with open(dest, "wb") as fh:
52         writer.write(fh)
53     return dest
54
55
56 def unlock(
57     path: Path | str,
58     candidates: Iterable[str] = (),
59     prompt_fn: Callable[[], str | None] | None = None,
60     max_prompts: int = 3,
61     dest: Path | str | None = None,
62 ) -> UnlockResult:
63     """Hybrid unlock: plaintext? -> candidate dictionary -> interactive prompt.
64
65     If dest is given and unlock succeeds, a decrypted copy is written there. The
66     working password is returned in the result but must never be logged.
67     """
68     src = Path(path)
69     if not is_encrypted(src):
70         out = Path(shutil.copyfile(src, dest)) if dest else None
71         return UnlockResult(True, "plaintext", None, out)
72
73     tried: set[str] = set()
74     for pw in candidates:
75         if pw in tried:
76             continue
77         tried.add(pw)
78         if password_works(src, pw):
79             out = write_decrypted(src, pw, dest) if dest else None
80             return UnlockResult(True, "candidate", pw, out)
81
82     if prompt_fn is not None:
83         for _ in range(max_prompts):
84             entered = prompt_fn()
85             if not entered:
86                 break
87             if password_works(src, entered):
88                 out = write_decrypted(src, entered, dest) if dest else None
89                 return UnlockResult(True, "prompt", entered, out)
90
91     return UnlockResult(False, None, None, None)
92

```

8. user (2026-06-10T16:13:40.365Z)

```

1     """Resolve a statement PDF's password for the ingest path ? the hybrid strategy, wired.
2
3     DESIGN.md §8.2 / ROADMAP C.1. Resolution order when a statement is encrypted:
4
5     1. **Candidate dictionary** ? passwords cached in the OS keychain (from earlier
successful
6     unlocks) plus those configured in ``password_rules.yaml`` (see
7     :mod:`muneem.password_rules`).
8     2. **Interactive prompt** ? a hidden ``getpass`` prompt (skipped with
``interactive=False``,
9     e.g. automation / CI).
10
11     A password that works **via the prompt** is cached in the keychain so the next ingest is
12     friction-free. Nothing here logs a password, writes one to the repo, or echoes it; the

```

```

cache is
13     the OS keychain (:mod:`muneem.secrets`), never a file.
14     ""
15
16     from __future__ import annotations
17
18     import getpass
19     import json
20     from pathlib import Path
21
22     from muneem import secrets
23     from muneem.errors import KeychainError
24     from muneem.password_rules import candidate_passwords
25     from muneem.unlock import is_encrypted, unlock
26
27     # Keychain entry holding a JSON list of known-good statement passwords (the learned
28     # candidates).
29     _CACHE_KEY = "statement_unlock_candidates"
30     # Bound the cached list so the keychain entry can't grow without limit; most-recently-
31     # used kept.
32     _MAX_CACHED = 20
33
34     def _cached_candidates() -> list[str]:
35         raw = secrets.get_secret(_CACHE_KEY)
36         if not raw:
37             return []
38         try:
39             return [p for p in json.loads(raw) if isinstance(p, str) and p]
40         except (ValueError, TypeError):
41             return []
42
43     def _remember(password: str) -> None:
44         """Cache a working password in the keychain (MRU order, deduped, capped). Never
45         logged.
46
47         Best-effort: a keychain write failure is swallowed (the password already unlocked
48         the file this
49         run); we never crash an ingest over a caching hiccup.
50         """
51         if not password or not password.strip():
52             return
53         current = [p for p in _cached_candidates() if p != password]
54         updated = [password, *current][:_MAX_CACHED] # most-recently-used first, bounded
55         try:
56             secrets.set_secret(_CACHE_KEY, json.dumps(updated))
57         except KeychainError:
58             pass
59
60     def resolve_password(pdf_path: Path | str, *, interactive: bool = True) -> str | None:
61         """Return a working password for ``pdf_path``.
62
63         ``""`` means the PDF is not encrypted (no password needed). ``None`` means it is
64         encrypted and
65         no candidate (or prompt) unlocked it. A prompt-resolved password is cached for next
66         time.
67         """
68         path = Path(pdf_path)
69         if not is_encrypted(path):
70             return ""
71
72         candidates = [*_cached_candidates(), *candidate_passwords()]
73         prompt_fn = None

```

```

71         if interactive:
72
73             def prompt_fn() -> str | None: # noqa: E306 ? local prompt; no echo, never
logged
74                 return getpass.getpass(f"Password for {path.name}: ") or None
75
76             result = unlock(path, candidates=candidates, prompt_fn=prompt_fn)
77             if not result.ok:
78                 return None
79             if result.method == "prompt" and result.password:
80                 _remember(result.password)
81             return result.password or ""
82

```

9. user (2026-06-10T16:13:40.845Z)

```

1     """Password rule loader for statement decryption (hybrid strategy support).
2
3     Prefers the user config dir (via platformdirs, e.g. ~/.config/muneem/password_rules.yaml
4     on Linux) for real rules. Falls back to legacy CWD "password_rules.yaml" (common during
5     dev) with a DeprecationWarning. **Never commit real rules or the file itself** ?
6     .gitignore`
7     blocks `password_rules.*`.
8
9     See DESIGN.md §8.2, CLAUDE.md security rules, and docs/ingesting-statements.md for the
file
10    format. Entries are either ``<bank>: <password>`` or a top-level ``candidates: [...]``
list.
11    """
12    from __future__ import annotations
13
14    import warnings
15    from pathlib import Path
16    from typing import Any
17
18    import yaml
19
20    from muneem.config import CONFIG_DIR
21    from muneem.errors import ConfigError
22
23
24    def _load_rules() -> dict[str, Any]:
25        """Load the rules dict from the config dir (preferred) or legacy CWD file; ``{}`` if
absent.
26
27        Resolution order:
28        1. ``<config_dir>/password_rules.yaml`` (recommended location)
29        2. ``./password_rules.yaml`` (legacy, emits DeprecationWarning)
30        """
31        for path, is_legacy in (
32            (CONFIG_DIR / "password_rules.yaml", False),
33            (Path("password_rules.yaml"), True),
34        ):
35            if path.exists():
36                if is_legacy:
37                    warnings.warn(
38                        "Loading password_rules from CWD (./password_rules.yaml) is
deprecated. "
39                        f"Move it to {CONFIG_DIR / 'password_rules.yaml'} for future-
proofing. "
40                        "See docs and CLAUDE.md (never commit real password material).",
41                        DeprecationWarning,
42                        stacklevel=3,
43                    )

```

```

44         with path.open(encoding="utf-8") as f:
45             try:
46                 data = yaml.safe_load(f)
47             except yaml.YAMLError as exc:
48                 # Never echo the file's contents in the error ? they are password
material.
49                 raise ConfigError(
50                     f"could not parse password rules ({path.name}): invalid YAML"
51                 ) from exc
52             if data is None:
53                 return {}
54             if not isinstance(data, dict):
55                 raise ConfigError(
56                     f"password rules ({path.name}) must be a mapping of <bank>:
<password>"
57                 )
58             return data
59         return {}
60
61
62     def get_password_for_bank(bank_name: str) -> str | None:
63         """Return the password (or derived rule string) for a bank, or ``None``."""
64         val = _load_rules().get(bank_name)
65         return str(val) if val else None
66
67
68     def candidate_passwords() -> list[str]:
69         """Every configured password to try as an unlock candidate, **bank-agnostic**.
70
71         The ingest path resolves the password *before* it knows the issuer (it needs the
decrypted
72         text to pick a parser), so it tries all configured passwords. Collects each per-bank
value
73         plus any entries under a top-level ``candidates`` list; order preserved, de-
duplicated.
74         Quote numeric passwords in the YAML so leading zeros survive.
75         """
76         rules = _load_rules()
77         collected: list[str] = []
78         for key, val in rules.items():
79             if key == "candidates" and isinstance(val, list):
80                 collected.extend(str(v) for v in val if v not in (None, ""))
81             elif isinstance(val, str | int | float):
82                 collected.append(str(val))
83         seen: set[str] = set()
84         unique: list[str] = []
85         for pw in collected:
86             if pw and pw not in seen:
87                 seen.add(pw)
88                 unique.append(pw)
89         return unique
90

```

10. user (2026-06-10T16:13:45.590Z)

```

1     """Local paths and configuration.
2
3     muneem is local-first: config lives in the OS config dir, data (DB + staging)
4     in the OS data dir. Nothing here touches the repository or the network, and no
5     secrets are stored here ? those go through :mod:`muneem.secrets` (the keychain).
6     """
7
8     from __future__ import annotations
9
10    import tomlib

```

```

11     from dataclasses import dataclass, field
12     from pathlib import Path
13     from typing import Any
14
15     from platformdirs import PlatformDirs
16
17     _dirs = PlatformDirs(appname="muneem", appauthor=False)
18
19     CONFIG_DIR = Path(_dirs.user_config_dir)
20     DATA_DIR = Path(_dirs.user_data_dir)
21     CONFIG_FILE = CONFIG_DIR / "config.toml"
22     DB_PATH = DATA_DIR / "muneem.db"
23     STAGING_DIR = DATA_DIR / "staging"
24
25
26     @dataclass(frozen=True)
27     class Config:
28         """Resolved local paths plus any values from ``config.toml``."""
29
30         config_dir: Path = CONFIG_DIR
31         data_dir: Path = DATA_DIR
32         config_file: Path = CONFIG_FILE
33         db_path: Path = DB_PATH
34         staging_dir: Path = STAGING_DIR
35         values: dict[str, Any] = field(default_factory=dict)
36
37
38     def load_config() -> Config:
39         """Load configuration, reading ``config.toml`` if it exists. Read-only."""
40         values: dict[str, Any] = {}
41         if CONFIG_FILE.is_file():
42             with CONFIG_FILE.open("rb") as fh:
43                 values = tomlib.load(fh)
44         return Config(values=values)
45
46
47     def ensure_dirs() -> None:
48         """Create the local config/data/staging dirs if missing. Never touches the repo."""
49         for directory in (CONFIG_DIR, DATA_DIR, STAGING_DIR):
50             directory.mkdir(parents=True, exist_ok=True)
51

```

11. user (2026-06-10T16:13:46.037Z)

```

1     """The ``muneem`` command-line interface.
2
3     Track A ships the promo spine: ``ingest`` a PDF into the local store and
4     ``offers list`` to read back what was found. ``--ocr`` adds local image OCR for
5     codes rasterized into banners (approximate; flagged for verification). Pipeline
6     subcommands (``fetch``, ``txns``) land with their milestones.
7     """
8
9     from __future__ import annotations
10
11     from contextlib import contextmanager
12     from pathlib import Path
13
14     import typer
15
16     from muneem import __version__
17     from muneem import db as dbmod
18     from muneem.config import load_config
19     from muneem.log import get_logger
20     from muneem.redaction import redact_filename
21

```

```

22     _log = get_logger(__name__)
23
24     app = typer.Typer(
25         add_completion=False,
26         help="muneem ? a local-first personal-finance bookkeeper.",
27         no_args_is_help=True,
28     )
29     offers_app = typer.Typer(no_args_is_help=True, help="Work with extracted offers.")
30     app.add_typer(offers_app, name="offers")
31
32     txns_app = typer.Typer(no_args_is_help=True, help="Work with parsed transactions.")
33     app.add_typer(txns_app, name="txns")
34
35     db_app = typer.Typer(no_args_is_help=True, help="Manage the local encrypted ledger.")
36     app.add_typer(db_app, name="db")
37
38
39     @contextmanager
40     def _ledger(db_path: Path):
41         """Open the encrypted ledger, turning an un-migrated *plaintext* file into a clean
CLI
42         error (pointing at ``muneem db encrypt``) rather than a raw traceback."""
43         try:
44             with dbmod.connect_encrypted(db_path) as conn:
45                 yield conn
46         except dbmod.EncryptedDatabaseError as exc:
47             typer.echo(str(exc), err=True)
48             raise typer.Exit(1) from exc
49
50
51     # Dispatch table for supported banks (extensible as new parsers land).
52     BANK_DISPATCH = {
53         "axis": (dbmod.list_axis_transactions, "debit", "credit"),
54         "axis_cc": (dbmod.list_axis_cc_transactions, "amount", "type"),
55         "au": (dbmod.list_au_transactions, "debit", "credit"),
56         "hdfc": (dbmod.list_hdfc_transactions, "withdrawal", "deposit"),
57     }
58
59
60     def _version_callback(value: bool) -> None:
61         if value:
62             typer.echo(f"muneem {__version__}")
63             raise typer.Exit()
64
65
66     @app.callback()
67     def main(
68         version: bool = typer.Option(
69             False,
70             "--version",
71             callback=_version_callback,
72             is_eager=True,
73             help="Show the muneem version and exit.",
74         ),
75     ) -> None:
76         """muneem command-line interface."""
77
78
79     @app.command()
80     def info() -> None:
81         """Show resolved local paths (no secrets, no sensitive data)."""
82         cfg = load_config()
83         present = "present" if cfg.config_file.is_file() else "absent"
84         typer.echo(f"muneem {__version__}")
85         typer.echo(f"config dir : {cfg.config_dir}")

```

```

86     typer.echo(f"data dir    : {cfg.data_dir}")
87     typer.echo(f"db path    : {cfg.db_path}")
88     typer.echo(f"staging    : {cfg.staging_dir}")
89     typer.echo(f"config file: {cfg.config_file} ({present})")
90
91
92     def _ingest_bank_statement(pdf: Path, db_path: Path, doc_type: str, *, interactive:
bool) -> None:
93         """Unlock (if encrypted) ? extract ? select parser ? store reconciled transactions.
94
95         The hybrid password unlock (cached/configured candidates ? prompt) runs *before*
96         extraction,
97         since the text needed to pick a parser is itself encrypted. The resolved password is
98         forwarded
99         to extraction and the parser, used in-memory only ? never logged.
100         """
101         from muneem import db as dbmod
102         from muneem.extract import extract
103         from muneem.parsers import get_parser
104         from muneem.statement_unlock import resolve_password
105         from muneem.unlock import is_encrypted
106
107         password: str | None = ""
108         if is_encrypted(pdf):
109             password = resolve_password(pdf, interactive=interactive)
110             if password is None:
111                 hint = " (run without --no-prompt to enter it)" if not interactive else ""
112                 typer.echo(f"Could not unlock {pdf.name}: no working password{hint}.",
err=True)
113                 raise typer.Exit(1)
114
115         ex = extract(pdf, password=password or "") # resolved above; `or ""` keeps the type
116         str
117         parser = get_parser(ex.text)
118         if parser is None:
119             typer.echo(f"No parser recognizes {pdf.name} as a supported bank statement.",
err=True)
120             raise typer.Exit(1)
121
122         with _ledger(db_path) as conn:
123             dbmod.init_db(conn)
124             existing = dbmod.get_document_by_sha(conn, ex.sha256)
125             if existing is not None:
126                 typer.echo(f"already ingested (doc {existing['id']}): {pdf.name}")
127                 return
128             doc_id = dbmod.insert_document(
129                 conn,
130                 source="file",
131                 external_id=pdf.name,
132                 doc_type=doc_type,
133                 sha256=ex.sha256,
134                 path=str(pdf.resolve()),
135                 status="parsed",
136             )
137             count = parser.parse_and_insert(pdf, conn, doc_id, password=password)
138             doc = dbmod.get_document_by_sha(conn, ex.sha256)
139             status = doc["status"] if doc else "parsed"
140
141         typer.echo(
142             f"ingested {pdf.name} (doc {doc_id}); parsed {count} transaction(s) "
143             f"via {parser.__class__.__name__} [{status}]"
144         )
145
146     def _eval_statement(pdf: Path, *, interactive: bool) -> tuple[str, int, str | None]:

```

```

145         """Run one statement through the full parse pipeline into an in-memory DB.
146
147     Returns `` (outcome, row_count, parser_name)`` where ``outcome`` is the document
status
148     (``reconciled`` / ``unreconciled``) or a failure tag (``locked`` / ``no-parser`` /
``error``).
149     Nothing is persisted and no statement value is surfaced ? only the verdict and
counts.
150     """
151     from muneem import db as dbmod
152     from muneem.extract import extract
153     from muneem.parsers import get_parser
154     from muneem.statement_unlock import resolve_password
155     from muneem.unlock import is_encrypted
156
157     password = ""
158     if is_encrypted(pdf):
159         password = resolve_password(pdf, interactive=interactive) or ""
160         if not password:
161             return ("locked", 0, None)
162     try:
163         ex = extract(pdf, password=password)
164         parser = get_parser(ex.text)
165     except Exception as exc: # eval must not crash on one bad file; record the redacted
reason
166         _log.warning(
167             "eval: extract/route failed for %s (%s)", redact_filename(pdf),
type(exc).__name__
168         )
169         return ("error", 0, None)
170     if parser is None:
171         return ("no-parser", 0, None)
172     name = parser.__class__.__name__
173     try:
174         with dbmod.connect(":memory:") as conn:
175             dbmod.init_db(conn)
176             doc_id = dbmod.insert_document(
177                 conn, source="file", external_id=pdf.name, sha256=ex.sha256,
status="parsed"
178             )
179             count = parser.parse_and_insert(pdf, conn, doc_id, password=password)
180             doc = dbmod.get_document_by_sha(conn, ex.sha256)
181             status = doc["status"] if doc else "parsed"
182     except Exception as exc:
183         _log.warning(
184             "eval: parse/store failed for %s (%s)", redact_filename(pdf),
type(exc).__name__
185         )
186         return ("error", 0, name)
187     return (status, count, name)
188
189
190 @app.command("eval")
191 def eval_statements(
192     folder: Path = typer.Argument(
193         ...,
194         exists=True,
195         file_okay=False,
196         readable=True,
197         help="A LOCAL folder of statement PDFs to evaluate (e.g. your seed or golden
set).",
198     ),
199     prompt: bool = typer.Option(
200         False,
201         "--prompt/--no-prompt",

```



```

202             help="Prompt for a password when cached/configured candidates don't unlock a
file.",
203         ),
204     ) -> None:
205         """Evaluate the parsers against a folder of statements and report the reconcile
rate.
206
207         The seed?golden onboarding loop (see docs/onboarding-a-bank.md): point it at a few
*seed*
208         statements while tuning a profile, then at a *golden* set to confirm. Everything
runs locally in
209         an in-memory DB; only counts / statuses / parser names print ? never statement
values.
210         """
211         pdfs = sorted(folder.glob("*.pdf"))
212         if not pdfs:
213             typer.echo(f"no .pdf files in {folder}")
214             return
215         reconciled = 0
216         for pdf in pdfs:
217             outcome, count, name = _eval_statement(pdf, interactive=prompt)
218             reconciled += outcome == "reconciled"
219             typer.echo(f" {outcome:12} {count:4d} row(s) {name or '?':20} {pdf.name}")
220         typer.echo(f"\n{reconciled}/{len(pdfs)} reconciled")
221
222
223 @app.command()
224 def ingest(
225     pdf: Path = typer.Argument(
226         ..., exists=True, dir_okay=False, readable=True, help="Path to a PDF to ingest."
227     ),
228     doc_type: str = typer.Option("promo", help="Document category."),
229     db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
230     ocr: bool = typer.Option(
231         False, "--ocr", help="Also OCR images locally (Tesseract) for codes in banners."
232     ),
233     no_prompt: bool = typer.Option(
234         False,
235         "--no-prompt",
236         help="Never prompt for a statement password; use only cached/configured
candidates.",
237     ),
238 ) -> None:
239     """Store a PDF's offers (promo) or transactions (bank_statement) in the local DB."""
240     from muneem.extract import extract
241     from muneem.offers import extract_offer
242
243     db_path = db or load_config().db_path
244
245     if doc_type == "bank_statement":
246         _ingest_bank_statement(pdf, db_path, doc_type, interactive=not no_prompt)
247         return
248
249     from muneem import db as dbmod
250
251     ex = extract(pdf)
252     text_fields = extract_offer(ex.text)
253     # (code, source) pairs; text-layer codes are exact, OCR codes are approximate.
254     found: list[tuple[str, str]] = [(c, "text") for c in text_fields.codes]
255     expiry_raw, expiry_type = text_fields.expiry_raw, text_fields.expiry_type
256
257     if ocr:
258         from muneem.ocr import ocr_pdf, tesseract_available
259

```

```

260         if not tesseract_available():
261             typer.echo("warning: --ocr requested but Tesseract not found; skipping
OCR.", err=True)
262         else:
263             ocr_fields = extract_offer(ocr_pdf(pdf))
264             known = {c for c, _ in found}
265             for code in ocr_fields.codes:
266                 if code not in known:
267                     found.append((code, "ocr"))
268                     known.add(code)
269             if expiry_raw is None:
270                 expiry_raw, expiry_type = ocr_fields.expiry_raw, ocr_fields.expiry_type
271
272     with _ledger(db_path) as conn:
273         dbmod.init_db(conn)
274         existing = dbmod.get_document_by_sha(conn, ex.sha256)
275         if existing is not None:
276             typer.echo(f"already ingested (doc {existing['id']}): {pdf.name}")
277             return
278         doc_id = dbmod.insert_document(
279             conn,
280             source="file",
281             external_id=pdf.name,
282             doc_type=doc_type,
283             sha256=ex.sha256,
284             path=str(pdf.resolve()),
285             status="parsed",
286         )
287         for code, src in found:
288             dbmod.insert_offer(
289                 conn,
290                 document_id=doc_id,
291                 promo_code=code,
292                 expiry_raw=expiry_raw,
293                 expiry_type=expiry_type,
294                 source=src,
295                 source_text=ex.text[:2000],
296             )
297
298         summary = ", ".join(f"{c} [{s}]" for c, s in found) if found else "none"
299         typer.echo(f"ingested {pdf.name} (doc {doc_id}); {len(found)} offer(s): {summary}")
300
301
302 @offers_app.command("list")
303 def offers_list(
304     db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
305 ) -> None:
306     """List extracted offers."""
307     from muneem import db as dbmod
308
309     db_path = db or load_config().db_path
310     if not Path(db_path).exists():
311         typer.echo("no offers yet (database not created).")
312         return
313
314     with _ledger(db_path) as conn:
315         dbmod.init_db(conn)
316         rows = dbmod.list_offers(conn)
317
318     if not rows:
319         typer.echo("no offers found.")
320         return
321     for row in rows:
322         who = row["merchant"] or row["external_id"] or "?"

```

```

323         when = f" ? expires {row['expiry_raw']}" if row["expiry_raw"] else ""
324         tag = " [ocr ? verify]" if row["source"] == "ocr" else ""
325         typer.echo(f"[{row['id']}] {who}: {row['promo_code']}{when}{tag}")
326
327
328     @txns_app.command("list")
329     def txns_list(
330         bank: str = typer.Argument(..., help="Which bank's transactions to list (e.g. hdfc,
axis)."),
331         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
332         limit: int = typer.Option(50, help="Max number of transactions to show."),
333     ) -> None:
334         """List parsed transactions for a specific bank."""
335         from muneem import db as dbmod
336
337         db_path = db or load_config().db_path
338         if not Path(db_path).exists():
339             typer.echo("database not created yet.")
340             return
341
342         bank = bank.lower()
343         if bank not in BANK_DISPATCH:
344             typer.echo(f"Unknown bank: {bank}", err=True)
345             return
346
347         list_fn, neg_key, pos_key = BANK_DISPATCH[bank]
348         with _ledger(db_path) as conn:
349             dbmod.init_db(conn)
350             rows = list_fn(conn, limit=limit)
351
352         if not rows:
353             typer.echo(f"no {bank} transactions found.")
354             return
355         for row in rows:
356             if bank == "axis_cc":
357                 amount = f"+{row[pos_key]}" if row["type"] == "Cr" else f"-{row[neg_key]}"
358             else:
359                 amount = f"+{row[pos_key]}" if row[pos_key] else f"-{row[neg_key]}"
360             part = (row["particulars"] or "")[:50]
361             typer.echo(f"[{row['account_number']}] {row['txn_date']} : {part:<50} |
{amount:>10}")
362
363
364     @db_app.command("status")
365     def db_status(
366         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
367     ) -> None:
368         """Show the ledger's encryption state (no statement data is read or printed)."""
369         from muneem import db as dbmod
370         from muneem.db_key import peek_key
371
372         db_path = db or load_config().db_path
373         key_state = "present in keychain" if peek_key() else "not yet minted"
374         if not Path(db_path).exists():
375             typer.echo(f"ledger : {db_path} (does not exist yet)")
376             typer.echo(f"key : {key_state}")
377             return
378         if dbmod.is_plaintext_sqlite(db_path):
379             state = "PLAINTEXT ? run 'muneem db encrypt' to harden it"
380         else:
381             state = "encrypted (SQLCipher)"
382         typer.echo(f"ledger : {db_path}")
383         typer.echo(f"state : {state}")

```

```

384         typer.echo(f"key      : {key_state}")
385
386
387     @db_app.command("encrypt")
388     def db_encrypt(
389         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
390 dir)."),
391     ) -> None:
392         """Migrate an existing *plaintext* ledger to an encrypted one (SQLCipher), in
393 place."""
394         from muneem import db as dbmod
395         from muneem.db_key import get_or_create_key
396
397         db_path = db or load_config().db_path
398         if not Path(db_path).exists():
399             typer.echo(
400                 f"nothing to encrypt: {db_path} does not exist (a new ledger is born
401 encrypted)."
402             )
403             return
404         if not dbmod.is_plaintext_sqlite(db_path):
405             typer.echo(
406                 f"{Path(db_path).name} is already encrypted (or not a SQLite file); nothing
407 to do."
408             )
409             return
410         n = dbmod.encrypt_database(db_path, get_or_create_key())
411         typer.echo(
412             f"encrypted {Path(db_path).name}: {n} table(s) migrated. A plaintext backup
413 remains at "
414             f"{Path(db_path).name}.plaintext.bak ? verify, then securely delete it."
415         )
416
417
418     def _require_encrypted_ledger(db_path: Path) -> str:
419         """Return the ledger's keychain key, or exit with a clear message if it can't be
420 used."""
421         from muneem import db as dbmod
422         from muneem.db_key import peek_key
423
424         if not Path(db_path).exists() or dbmod.is_plaintext_sqlite(db_path):
425             typer.echo("no encrypted ledger here (run 'muneem db encrypt' first).",
426 err=True)
427             raise typer.Exit(1)
428         key = peek_key()
429         if not key:
430             typer.echo("no encryption key in the keychain for this ledger.", err=True)
431             raise typer.Exit(1)
432         return key
433
434
435     @db_app.command("rotate-key")
436     def db_rotate_key(
437         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
438 dir)."),
439     ) -> None:
440         """Rotate the ledger's encryption key (SQLCipher rekey) and update the keychain."""
441         from muneem import db as dbmod
442         from muneem import db_key
443
444         db_path = db or load_config().db_path
445         current = _require_encrypted_ledger(db_path)
446         fresh = db_key.new_key()
447         # Keychain-first with rollback: if the file rekey fails, the keychain still matches
448 the file.

```

```

440     db_key.set_key(fresh)
441     try:
442         dbmod.rekey_database(db_path, current, fresh)
443     except dbmod.EncryptedDatabaseError as exc:
444         db_key.set_key(current) # roll back; the file is still under the old key
445         typer.echo(str(exc), err=True)
446         raise typer.Exit(1) from exc
447     typer.echo(f"rotated the encryption key for {Path(db_path).name}.")
448
449
450     @db_app.command("backup")
451     def db_backup(
452         dest: Path = typer.Argument(..., help="Destination path for the encrypted backup
copy."),
453         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
454     ) -> None:
455         """Write an encrypted online-backup copy of the ledger (consistent even with WAL
active)."""
456         from muneem import db as dbmod
457
458         db_path = db or load_config().db_path
459         key = _require_encrypted_ledger(db_path)
460         try:
461             dbmod.backup_database(db_path, dest, key)
462         except dbmod.EncryptedDatabaseError as exc:
463             typer.echo(str(exc), err=True)
464             raise typer.Exit(1) from exc
465         typer.echo(f"backed up {Path(db_path).name} to {dest} (encrypted).")
466
467
468     @db_app.command("setup-recovery")
469     def db_setup_recovery(
470         out: Path | None = typer.Option(
471             None, help="Recovery file path (default: <data dir>/muneem-recovery.json)."
472         ),
473     ) -> None:
474         """Create a passphrase-wrapped recovery file for the ledger's encryption key.
475
476         Lets you restore the key (and thus the ledger + backups) if the OS keychain is ever
lost. The
477         file holds only the *wrapped* key ? useless without the passphrase.
478         """
479         import getpass
480
481         from muneem import recovery
482         from muneem.db_key import peek_key
483
484         key = peek_key()
485         if not key:
486             typer.echo("no encryption key in the keychain yet (create the ledger first).",
err=True)
487             raise typer.Exit(1)
488         out_path = out or (load_config().data_dir / "muneem-recovery.json")
489         passphrase = getpass.getpass("New recovery passphrase: ")
490         if passphrase != getpass.getpass("Confirm passphrase: "):
491             typer.echo("passphrases did not match.", err=True)
492             raise typer.Exit(1)
493         try:
494             recovery.write_recovery_file(out_path, key, passphrase)
495         except recovery.RecoveryError as exc:
496             typer.echo(str(exc), err=True)
497             raise typer.Exit(1) from exc
498         typer.echo(
499             f"wrote recovery file to {out_path}. Store it OFF this machine (password manager

```

```

/ "
500         "separate drive) and remember the passphrase ? you need BOTH to recover. Without
them, a "
501         "lost keychain means the ledger and its backups are unrecoverable."
502     )
503
504
505     @db_app.command("recover")
506     def db_recover(
507         file: Path = typer.Argument(..., exists=True, dir_okay=False, help="Recovery file
path."),
508         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
509     ) -> None:
510         """Restore the ledger's encryption key from a recovery file + passphrase (into the
keychain)."""
511         import getpass
512
513         from muneem import db as dbmod
514         from muneem import db_key, recovery
515
516         passphrase = getpass.getpass("Recovery passphrase: ")
517         try:
518             key = recovery.read_recovery_file(file, passphrase)
519         except recovery.RecoveryError as exc:
520             typer.echo(str(exc), err=True)
521             raise typer.Exit(1) from exc
522         db_path = db or load_config().db_path
523         # If the ledger is present, verify the recovered key actually opens it before
committing it.
524         if Path(db_path).exists() and not dbmod.is_plaintext_sqlite(db_path):
525             try:
526                 with dbmod.connect(db_path, key=key) as conn:
527                     conn.execute("SELECT 1")
528             except dbmod.EncryptedDatabaseError as exc:
529                 typer.echo("recovered a key, but it does not open the current ledger.",
err=True)
530                 raise typer.Exit(1) from exc
531         db_key.set_key(key)
532         typer.echo("recovered the encryption key into the keychain.")
533
534
535     if __name__ == "__main__":
536         app()
537

```

12. user (2026-06-10T16:13:46.990Z)

```

1     """muneem's exception hierarchy.
2
3     Every error muneem raises derives from :class:`MuneemError`, so callers can catch the
whole family
4     with one `except`. Two rules for messages on these exceptions, because they may end up
in logs:
5
6     1. **Only non-sensitive context.** A file *name* is acceptable (it's the user's own
machine), but
7         never raw statement text, passwords, balances, or account numbers. Use
8         :mod:`muneem.redaction` helpers to sanitise anything derived from a document.
9     2. **Chain the cause.** Raise with ``raise ParseError(...) from exc`` so the original
traceback is
10        available for local debugging while the muneem-level message stays clean.
11    """
12
13    from __future__ import annotations

```

```

14
15
16     class MuneemError(Exception):
17         """Base class for all errors raised by muneem."""
18
19
20     class ParseError(MuneemError):
21         """A document could not be parsed (corrupt PDF, unexpected layout, extraction
failure).
22
23         Carries only non-sensitive context (e.g. the file name and the failing stage) ?
never the
24         statement contents or the underlying library's message verbatim.
25         """
26
27
28     class UnlockError(MuneemError):
29         """A password-protected document could not be unlocked (no working password, bad
input)."""
30
31
32     class EncryptedDatabaseError(MuneemError):
33         """The encrypted ledger could not be opened ? wrong key, missing SQLCipher binding,
or a
34         plaintext file where an encrypted one was expected."""
35
36
37     class KeychainError(MuneemError):
38         """Storing a secret in the OS keychain failed (no backend, locked keychain, etc.).
39
40         The secret *name* is safe to include; never the value."""
41
42
43     class ConfigError(MuneemError):
44         """A user config file (e.g. password_rules.yaml) is malformed.
45
46         Messages must reference the file by name only ? never echo its contents, which may
hold
47         password material."""
48
49
50     class MigrationError(MuneemError):
51         """A schema migration could not be applied, or the on-disk schema diverged from the
52         migration history (e.g. the frozen baseline was edited without adding a
migration)."""
53

```

13. user (2026-06-10T16:13:47.165Z)

```

1     """Central, redaction-aware logging.
2
3     Use ``get_logger(__name__)`` instead of the stdlib ``logging.getLogger`` so every muneem
logger
4     carries a :class:`RedactingFilter`. The filter scrubs money-/account-like substrings
from each
5     record as a backstop ? code should still pass already-sanitised context (see
6     :mod:`muneem.redaction`); this is defense-in-depth so an accidental sensitive f-string
cannot leak.
7
8     muneem stays library-style: we attach the filter but do not configure handlers or
levels (the
9     application / CLI owns that), so importing muneem never hijacks the root logger.
10    """
11
12    from __future__ import annotations

```

```

13
14     import logging
15
16     from muneem.redaction import redact_text
17
18
19     class RedactingFilter(logging.Filter):
20         """A logging filter that masks sensitive substrings in the formatted message."""
21
22         def filter(self, record: logging.LogRecord) -> bool:
23             try:
24                 message = record.getMessage()
25             except Exception: # never drop a record just because formatting failed
26                 return True
27             redacted = redact_text(message)
28             if redacted != message:
29                 record.msg = redacted
30                 record.args = ()
31             return True
32
33
34     _filtered: set[str] = set()
35
36
37     def get_logger(name: str) -> logging.Logger:
38         """Return a logger that redacts sensitive substrings from its records
39         (idempotent)."""
40         logger = logging.getLogger(name)
41         if name not in _filtered:
42             logger.addFilter(RedactingFilter())
43             _filtered.add(name)
44         return logger

```

14. user (2026-06-10T16:13:53.463Z)

```

1     """Redaction helpers ? keep sensitive data out of logs, exceptions, and stdout.
2
3     muneem aggregates the user's whole financial life, so a balance, account number, or
4     password that
5     slips into a log line or an exception message is a real incident. These helpers produce
6     *safe*
7     representations for diagnostics. They are deliberately conservative: it is better to
8     over-mask a
9     diagnostic string than to leak one.
10    """
11
12    from __future__ import annotations
13
14    import re
15    from pathlib import Path
16
17    # Long digit runs look like account / card / customer numbers.
18    _ACCOUNT_RE = re.compile(r"\d{9,}")
19    # Money-like: Indian or Western grouping with a decimal, or a plain N.NN amount.
20    _AMOUNT_RE = re.compile(r"\d{1,3}(?:,\d{2,3})+(?:\.\d{1,2})?|\d+\.\d{2}")
21    # Digit runs in a filename that may be PII (DOB, account suffix, customer id).
22    _FILENAME_DIGITS_RE = re.compile(r"\d{4,}")
23
24
25    def redact_filename(path: str | Path) -> str:
26        """Return a file's *name only* (never the full path), with long digit runs masked.
27
28        The full path can expose the user's home/account layout, and a bare basename can
29        still carry PII

```



```

26         (Indian statement names are often ``Name_DDMMYYYY.pdf``), so digit runs of 4+ are
masked. Keeps
27         enough of the name to be useful in a diagnostic.
28         """
29         name = Path(str(path)).name
30         return _FILENAME_DIGITS_RE.sub(lambda m: "#" * len(m.group()), name)
31
32
33     def redact_text(text: str) -> str:
34         """Mask account-number-like and money-like substrings in free text (e.g. before
logging).
35
36         A backstop for accidental leaks ? code should already pass sanitised context; this
catches what
37         slips through. Returns the input unchanged if there is nothing to mask.
38         """
39         if not text:
40             return text
41         masked = _AMOUNT_RE.sub("?amt?", text)
42         masked = _ACCOUNT_RE.sub("?acct?", masked)
43         return masked
44
45
46     def mask_secret(secret: str | None) -> str:
47         """A non-reversible placeholder for a password/key ? its length only, never the
value."""
48         if not secret:
49             return "?none?"
50         return f"?secret:{len(secret)} chars?"
51

```

15. user (2026-06-10T16:13:53.946Z)

```

1     """PDF text extraction (the deterministic, born-digital path).
2
3     Uses **pdfminer.six (MIT)** ? a permissive, commercially-safe, pure-Python text
4     extractor. We deliberately do not use PyMuPDF/``fitz``: it is AGPL-3.0,
5     incompatible with muneem's commercial-safety principle (see docs/DEPENDENCIES.md).
6
7     NFKC normalization is applied because the real corpus returns ligatures
8     (e.g. U+FB00 '?') that would otherwise break naive matching. Image extraction +
9     OCR (for codes rasterized into banners, e.g. Swiggy's CTSWIGGYUX) is the image
10    tier and lands with the OCR work (pypdfium2 + Tesseract, both permissive).
11    """
12
13    from __future__ import annotations
14
15    import hashlib
16    import unicodedata
17    from dataclasses import dataclass
18    from pathlib import Path
19
20    from pdfminer.high_level import extract_text
21
22
23    @dataclass
24    class Extracted:
25        """The result of reading a PDF's text layer."""
26
27        path: Path
28        sha256: str
29        text: str
30
31
32    def _normalize(text: str) -> str:

```

```

33     """NFKC-normalize so ligatures fold to ASCII (U+FB00 '?' -> 'ff')."""
34     return unicodedata.normalize("NFKC", text)
35
36
37 def file_sha256(path: Path | str) -> str:
38     """Stream the file and return its hex SHA-256 (used for document dedup)."""
39     digest = hashlib.sha256()
40     with open(path, "rb") as fh:
41         for chunk in iter(lambda: fh.read(65536), b''):
42             digest.update(chunk)
43     return digest.hexdigest()
44
45
46 def extract(pdf_path: Path | str, password: str = "") -> Extracted:
47     """Extract and NFKC-normalize the full text layer of a PDF.
48
49     ``password`` is forwarded to pdftminer for encrypted statements and used in-memory
50 only ? it is
51     never stored or logged. The sha256 is over the file bytes, so it is independent of
52 the password.
53 """
54     path = Path(pdf_path)
55     text = extract_text(str(path), password=password or "")
56     return Extracted(path=path, sha256=file_sha256(path), text=_normalize(text))

```

16. user (2026-06-10T16:13:54.431Z)

```

1     """Local OCR for codes rasterized into banner images (the image tier).
2
3     100% local and permissive ? pypdfium2 (Apache/BSD) renders pages, Tesseract
4     (Apache-2.0) reads them. No paid vision API (see DESIGN.md § 5 rule 9).
5
6     Grounded on the corpus: grayscale render at ~288 DPI (scale 4.0) is what reliably
7     reads stylized banner text. Note OCR is approximate ? e.g. Swiggy's white-on-orange
8     boxed `CTSWIGGYUX` reads as `ICTSWIGGYUX` (a leading box-edge artifact). Callers
9     should treat OCR-derived codes as low-confidence ("verify"), and matching against
10    known codes is fuzzy, never exact (docs/TESTING.md § 6). A purpose-trained model
11    is the eventual upgrade for stylized codes.
12    """
13
14    from __future__ import annotations
15
16    import os
17    import shutil
18    import unicodedata
19    from pathlib import Path
20
21    import pypdfium2 as pdfium
22    import pytesseract
23    from PIL import ImageOps
24
25    # Common install locations, tried only if Tesseract isn't on PATH.
26    _DEFAULT_TESSERACT_PATHS = (
27        r"C:\Program Files\Tesseract-OCR\tesseract.exe", # Windows (winget / UB-Mannheim)
28        "/usr/bin/tesseract", # Linux (apt / dnf)
29        "/usr/local/bin/tesseract", # Linux from source / Intel Homebrew
30        "/opt/homebrew/bin/tesseract", # macOS Apple-silicon Homebrew
31    )
32
33
34    def _resolve_tesseract() -> str | None:
35        """Find the Tesseract binary: env override, then PATH, then common OS locations."""
36        cmd = os.environ.get("MUNEEM_TESSERACT_CMD")
37        if cmd and Path(cmd).exists():

```

```

38         return cmd
39     found = shutil.which("tesseract")
40     if found:
41         return found
42     for candidate in _DEFAULT_TESSERACT_PATHS:
43         if Path(candidate).exists():
44             return candidate
45     return None
46
47
48 def tesseract_available() -> bool:
49     return _resolve_tesseract() is not None
50
51
52 def ocr_pdf(pdf_path: Path | str, scale: float = 4.0) -> str:
53     """Render each page locally and OCR it; returns NFKC-normalized text."""
54     cmd = _resolve_tesseract()
55     if cmd is None:
56         raise RuntimeError(
57             "Tesseract not found. Install it, or set MUNEEM_TESSERACT_CMD to its path."
58         )
59     pytesseract.pytesseract.tesseract_cmd = cmd
60     pdf = pdfium.PdfDocument(str(pdf_path))
61     parts: list[str] = []
62     try:
63         for page in pdf:
64             image = ImageOps.grayscale(page.render(scale=scale).to_pil())
65             parts.append(pytesseract.image_to_string(image))
66     finally:
67         pdf.close()
68     return unicodedata.normalize("NFKC", "\n".join(parts))
69

```

17. user (2026-06-10T16:13:54.919Z)

```

1     """Deterministic promo-offer extraction from normalized email text.
2
3     Precision-first: we anchor on an explicit "code" cue and validate the captured
4     token, so a promo email with *no* code (e.g. Flipkart's GOAT-sale banner)
5     correctly yields nothing rather than a hallucinated code. Codes rasterized into
6     images (e.g. Swiggy) are out of scope here ? that's the OCR tier.
7     """
8
9     from __future__ import annotations
10
11     import re
12     from dataclasses import dataclass
13
14     _CODE_RE = re.compile(
15         r"\b(?:coupon\s+code|promo\s*code|use\s+code|using\s+code|code)\b\s*[:\-\]?\s*([A-Za-
16         z0-9]{4,20})",
17         re.IGNORECASE,
18     )
19
20     _EXPIRY_PATTERNS: list[tuple[re.Pattern[str], str]] = [
21         (re.compile(r"\bends today\b", re.I), "relative"),
22         (re.compile(r"\blast few hours\b", re.I), "relative"),
23         (re.compile(r"\bfinal day\b", re.I), "relative"),
24         (re.compile(r"\bbefore\s+[A-Z][a-z]+\s+\d{1,2}\w*", re.I), "absolute"),
25         (re.compile(r"\bvalid\s+(?:till|until|through)\b[^\n]{0,40}", re.I), "absolute"),
26         (re.compile(r"\bexpir\w*\b[^\n]{0,40}", re.I), "absolute"),
27     ]
28
29     def _looks_like_code(token: str) -> bool:

```

```

30     """A code-like token has a digit, or is all-caps and reasonably long."""
31     if any(ch.isdigit() for ch in token):
32         return True
33     return token.isupper() and len(token) >= 5
34
35
36     @dataclass
37     class OfferFields:
38         codes: list[str]
39         expiry_raw: str | None
40         expiry_type: str | None
41
42
43     def extract_offer(text: str) -> OfferFields:
44         """Pull promo code(s) and a best-effort expiry phrase from email text."""
45         codes: list[str] = []
46         for match in _CODE_RE.finditer(text):
47             token = match.group(1)
48             if _looks_like_code(token) and token not in codes:
49                 codes.append(token)
50
51         expiry_raw: str | None = None
52         expiry_type: str | None = None
53         for pattern, etype in _EXPIRY_PATTERNS:
54             found = pattern.search(text)
55             if found:
56                 expiry_raw = found.group(0).strip()
57                 expiry_type = etype
58                 break
59
60         return OfferFields(codes=codes, expiry_raw=expiry_raw, expiry_type=expiry_type)
61

```

18. assistant (2026-06-10T16:16:48.734Z)

Now let me check logger adoption across the package, the tests, and a few git/tracking concerns.

19. user (2026-06-10T16:16:49.786Z)

```

src\muneem\cli.py:19:from muneem.log import get_logger
src\muneem\cli.py:22:_log = get_logger(__name__)
src\muneem\log.py:3:Use ``get_logger(__name__)`` instead of the stdlib ``logging.getLogger`` so
every muneem logger
src\muneem\log.py:14:import logging
src\muneem\log.py:37:def get_logger(name: str) -> logging.Logger:
src\muneem\log.py:39:    logger = logging.getLogger(name)

```

20. user (2026-06-10T16:16:50.740Z)

```

src\muneem\__init__.py
src\muneem\config.py
src\muneem\offers.py
src\muneem\ocr.py
src\muneem\ingestion\__init__.py
src\muneem\ingestion\base.py
src\muneem\ingestion\gmail.py
src\muneem\parsers\__init__.py
src\muneem\parsers\concepts.py
src\muneem\parsers\validation.py
src\muneem\parsers\clustering.py
src\muneem\parsers\profiles.py
src\muneem\extract.py
src\muneem\parsers\savings.py
src\muneem\log.py

```

```
src\muneem\redaction.py
src\muneem\password_rules.py
src\muneem\secrets.py
src\muneem\statement_unlock.py
src\muneem\parsers\engine.py
src\muneem\unlock.py
src\muneem\db_key.py
src\muneem\parsers\base.py
src\muneem\ingestion\imap.py
src\muneem\ingestion\outlook.py
src\muneem\cli.py
src\muneem\recovery.py
src\muneem\errors.py
src\muneem\db.py
src\muneem\migrations.py
src\muneem\parsers\au.py
src\muneem\parsers\axis.py
src\muneem\parsers\hdfc.py
```

21. user (2026-06-10T16:16:51.109Z)

```
tests\test_smoke.py
tests\test_db.py
tests\test_extract.py
tests\test_promo_regression.py
tests\test_ocr.py
tests\test_unlock.py
tests\test_offers.py
tests\test_repo_hygiene.py
tests\synthpdf.py
tests\test_regression_synth.py
tests\test_eval_cli.py
tests\conftest.py
tests\test_cli.py
tests\test_db_crypto.py
tests\test_errors.py
tests\test_log.py
tests\test_redaction.py
tests\test_password_rules.py
tests\test_secrets.py
tests\test_statement_unlock.py
tests\test_db_crypto_ops.py
tests\test_verifier_properties.py
tests\test_dependency_licenses.py
tests\test_precommit_hook.py
tests\test_recovery.py
tests\test_db_invariants.py
tests\test_migrations.py
tests\test_provenance.py
```

22. user (2026-06-10T16:16:59.724Z)

```
1      """At-rest encryption for the local ledger (SQLCipher via sqlcipher3).
2
3      Covers the safety-critical invariants of the encryption seam:
4      - a keyed open round-trips data through SQLCipher with SQL queries intact;
5      - a wrong key is rejected (never silently returns garbage / empty);
6      - the on-disk file is genuinely encrypted (stdlib sqlite3 cannot read it);
7      - ``:memory:`` and keyless opens stay plaintext (the test / CI path);
8      - the keychain-backed key is minted once and reused;
9      - a pre-existing *plaintext* ledger is refused by ``connect_encrypted`` and can be
10        migrated in place by ``encrypt_database``;
11      - the key never appears in an error message.
12
13      No real statement data is involved ? only fabricated numbers.
```

```

14     """
15
16     from __future__ import annotations
17
18     import sqlite3
19
20     import pytest
21
22     pytest.importorskip("sqlcipher3") # encryption backend; a hard dep, skip only if truly
absent
23
24     from muneem import db as dbmod # noqa: E402 ? after importorskip by design
25
26     KEY = "00112233445566778899aabbccddeeff00112233445566778899aabbccddeeff"
27     WRONG = "ffeeddccbbaa99887766554433221100ffeeddccbbaa99887766554433221100"
28
29
30     def _fake_keychain(monkeypatch):
31         """Replace the OS keychain with an in-memory dict for deterministic, offline
tests."""
32         store: dict[str, str] = {}
33
34         def _set(name, value):
35             store[name] = value
36
37         monkeypatch.setattr("muneem.secrets.get_secret", lambda name: store.get(name))
38         monkeypatch.setattr("muneem.secrets.set_secret", _set)
39         return store
40
41
42     def test_encrypted_roundtrip_keeps_sql_queryability(tmp_path):
43         p = tmp_path / "led.db"
44         with dbmod.connect(p, key=KEY) as conn:
45             conn.execute("CREATE TABLE t (id INTEGER PRIMARY KEY, amount REAL, note TEXT)")
46             conn.executemany(
47                 "INSERT INTO t (amount, note) VALUES (?, ?)", [(100.5, "a"), (250.0, "b"),
(75.25, "c")]
48             )
49         # Reopen with the same key; WHERE / SUM / ORDER BY all work through the encryption.
50         with dbmod.connect(p, key=KEY) as conn:
51             total = conn.execute("SELECT SUM(amount) FROM t WHERE amount >
80").fetchone()[0]
52             assert total == pytest.approx(350.5)
53             rows = conn.execute("SELECT amount FROM t ORDER BY amount").fetchall()
54             assert [r["amount"] for r in rows] == [75.25, 100.5, 250.0]
55
56
57     def test_wrong_key_is_rejected(tmp_path):
58         p = tmp_path / "led.db"
59         with dbmod.connect(p, key=KEY) as conn:
60             conn.execute("CREATE TABLE t (id INTEGER PRIMARY KEY, x REAL)")
61             conn.execute("INSERT INTO t (x) VALUES (1.0)")
62         with pytest.raises(dbmod.EncryptedDatabaseError):
63             with dbmod.connect(p, key=WRONG):
64                 pass # the wrong key is caught at open (key verification), before any query
65
66
67     def test_on_disk_file_is_actually_encrypted(tmp_path):
68         p = tmp_path / "led.db"
69         with dbmod.connect(p, key=KEY) as conn:
70             conn.execute("CREATE TABLE t (id INTEGER PRIMARY KEY, x REAL)")
71             conn.execute("INSERT INTO t (x) VALUES (1.0)")
72         # Not a plaintext SQLite file, and stock stdlib sqlite3 cannot open it.
73         assert not dbmod.is_plaintext_sqlite(p)
74         con = sqlite3.connect(p)

```

```

75         try:
76             with pytest.raises(sqlite3.DatabaseError):
77                 con.execute("SELECT * FROM t").fetchone()
78         finally:
79             con.close()
80
81
82     def test_memory_and_keyless_paths_stay_plaintext(tmp_path):
83         # :memory: ignores keys and uses stdlib sqlite3.
84         with dbmod.connect(":memory:") as conn:
85             assert isinstance(conn, sqlite3.Connection)
86             conn.execute("CREATE TABLE t (id INTEGER)")
87         # A keyless file open is a normal plaintext SQLite file.
88         p = tmp_path / "plain.db"
89         with dbmod.connect(p) as conn:
90             conn.execute("CREATE TABLE t (id INTEGER)")
91         assert dbmod.is_plaintext_sqlite(p)
92
93
94     def test_get_or_create_key_is_idempotent_and_64_hex(monkeypatch):
95         _fake_keychain(monkeypatch)
96         from muneem import db_key
97
98         assert db_key.peek_key() is None
99         k1 = db_key.get_or_create_key()
100        assert len(k1) == 64 and all(c in "0123456789abcdef" for c in k1)
101        k2 = db_key.get_or_create_key()
102        assert k1 == k2 # not re-minted
103        assert db_key.peek_key() == k1
104
105
106     def test_connect_encrypted_mints_key_and_roundtrips(tmp_path, monkeypatch):
107         _fake_keychain(monkeypatch)
108         p = tmp_path / "led.db"
109         with dbmod.connect_encrypted(p) as conn:
110             conn.execute("CREATE TABLE t (id INTEGER PRIMARY KEY, x REAL)")
111             conn.execute("INSERT INTO t (x) VALUES (7.0)")
112         assert not dbmod.is_plaintext_sqlite(p) # written encrypted
113         with dbmod.connect_encrypted(p) as conn:
114             assert conn.execute("SELECT x FROM t").fetchone()["x"] == pytest.approx(7.0)
115
116
117     def test_connect_encrypted_refuses_a_plaintext_ledger(tmp_path, monkeypatch):
118         _fake_keychain(monkeypatch)
119         p = tmp_path / "led.db"
120         con = sqlite3.connect(p) # create a plaintext DB the old way
121         con.execute("CREATE TABLE t (id INTEGER)")
122         con.commit()
123         con.close()
124         with pytest.raises(dbmod.EncryptedDatabaseError):
125             with dbmod.connect_encrypted(p):
126                 pass
127
128
129     def test_encrypt_database_migrates_in_place(tmp_path):
130         p = tmp_path / "led.db"
131         con = sqlite3.connect(p)
132         con.execute("CREATE TABLE t (id INTEGER PRIMARY KEY, amount REAL)")
133         con.execute("INSERT INTO t (amount) VALUES (42.0)")
134         con.commit()
135         con.close()
136         assert dbmod.is_plaintext_sqlite(p)
137
138         n = dbmod.encrypt_database(p, KEY)
139         assert n >= 1

```

```

140         assert not dbmod.is_plaintext_sqlite(p) # now encrypted in place
141         assert (tmp_path / "led.db.plaintext.bak").exists() # original preserved for
verification
142         with dbmod.connect(p, key=KEY) as conn:
143             assert conn.execute("SELECT amount FROM t").fetchone()["amount"] ==
pytest.approx(42.0)
144
145
146     def test_error_message_never_leaks_the_key(tmp_path):
147         p = tmp_path / "led.db"
148         with dbmod.connect(p, key=KEY) as conn:
149             conn.execute("CREATE TABLE t (id INTEGER PRIMARY KEY)")
150         try:
151             with dbmod.connect(p, key=WRONG):
152                 pass
153         except dbmod.EncryptedDatabaseError as exc:
154             assert KEY not in str(exc) and WRONG not in str(exc)
155

```

23. user (2026-06-10T16:16:59.980Z)

```

1     """Crypto operations on the encrypted ledger: key rotation, encrypted backup, migration
temp
2     cleanup, correctness at size, and the plaintext-refusal guidance ? plus their CLI
commands.
3
4     Offline, fabricated data only; the autouse conftest fixture keeps the real keychain
untouched.
5     """
6
7     import sqlite3
8
9     import pytest
10
11     pytest.importorskip("sqlcipher3")
12
13     from typer.testing import CliRunner # noqa: E402
14
15     from muneem import db as dbmod # noqa: E402
16     from muneem import db_key # noqa: E402
17     from muneem.cli import app # noqa: E402
18
19     runner = CliRunner()
20
21     KEY_A = "00112233445566778899aabbccddeeff00112233445566778899aabbccddeeff"
22     KEY_B = "ffeeddccbbaa99887766554433221100ffeeddccbbaa99887766554433221100"
23
24
25     def _seed_encrypted(path, key, n=3):
26         with dbmod.connect(path, key=key) as conn:
27             conn.execute("CREATE TABLE t (id INTEGER PRIMARY KEY, amount REAL)")
28             conn.executemany("INSERT INTO t (amount) VALUES (?)", [(float(i),) for i in
range(n)])
29
30
31     # --- rekey / rotation
-----
32
33
34     def test_rekey_changes_the_key_and_keeps_data(tmp_path):
35         p = tmp_path / "led.db"
36         _seed_encrypted(p, KEY_A, n=5)
37         dbmod.rekey_database(p, KEY_A, KEY_B)
38         with pytest.raises(dbmod.EncryptedDatabaseError): # old key no longer opens it
39             with dbmod.connect(p, key=KEY_A):

```



```

40         pass
41     with dbmod.connect(p, key=KEY_B) as conn: # new key works, data intact
42         assert conn.execute("SELECT count(*) FROM t").fetchone()[0] == 5
43
44
45     def test_rekey_with_wrong_current_key_raises(tmp_path):
46         p = tmp_path / "led.db"
47         _seed_encrypted(p, KEY_A)
48         with pytest.raises(dbmod.EncryptedDatabaseError):
49             dbmod.rekey_database(p, KEY_B, KEY_A)
50
51
52     # --- encrypted backup
53     -----
54
55     def test_backup_is_encrypted_and_intact(tmp_path):
56         p = tmp_path / "led.db"
57         _seed_encrypted(p, KEY_A, n=4)
58         dest = tmp_path / "backup" / "led.bak.db"
59         dbmod.backup_database(p, dest, KEY_A)
60         assert dest.exists() and not dbmod.is_plaintext_sqlite(dest) # backup stays
encrypted
61         with dbmod.connect(dest, key=KEY_A) as conn:
62             assert conn.execute("SELECT count(*) FROM t").fetchone()[0] == 4
63         with pytest.raises(dbmod.EncryptedDatabaseError): # wrong key can't read the backup
64             with dbmod.connect(dest, key=KEY_B):
65                 pass
66
67
68     def test_backup_with_wrong_key_raises(tmp_path):
69         p = tmp_path / "led.db"
70         _seed_encrypted(p, KEY_A)
71         with pytest.raises(dbmod.EncryptedDatabaseError):
72             dbmod.backup_database(p, tmp_path / "b.db", KEY_B)
73
74
75     # --- migration cleanup + correctness at size + plaintext refusal
76     -----
77
78     def test_encrypt_database_cleans_temp_on_failure(tmp_path, monkeypatch):
79         p = tmp_path / "led.db"
80         con = sqlite3.connect(p)
81         con.execute("CREATE TABLE t (id INTEGER PRIMARY KEY)")
82         con.commit()
83         con.close()
84
85         def boom(*_a, **_k): # fail the post-export verification open
86             raise RuntimeError("boom during verify")
87
88         monkeypatch.setattr(dbmod, "_open_encrypted", boom)
89         with pytest.raises(RuntimeError):
90             dbmod.encrypt_database(p, KEY_A)
91         assert not (tmp_path / "led.db.enc.tmp").exists() # partial temp removed
92         assert dbmod.is_plaintext_sqlite(p) # original left untouched
93
94
95     def test_encrypted_ledger_handles_a_larger_table(tmp_path):
96         p = tmp_path / "led.db"
97         with dbmod.connect(p, key=KEY_A) as conn:
98             conn.execute("CREATE TABLE t (id INTEGER PRIMARY KEY, amount REAL)")
99             conn.executemany("INSERT INTO t (amount) VALUES (?)", [(float(i),) for i in
range(2000)])
100         with dbmod.connect(p, key=KEY_A) as conn:

```

```

101         total = conn.execute("SELECT SUM(amount) FROM t").fetchone()[0]
102         cnt = conn.execute("SELECT count(*) FROM t WHERE amount >= 1000").fetchone()[0]
103         assert cnt == 1000 and total == float(sum(range(2000)))
104
105
106     def test_connect_encrypted_plaintext_message_points_to_db_encrypt(tmp_path):
107         p = tmp_path / "led.db"
108         con = sqlite3.connect(p)
109         con.execute("CREATE TABLE t (id INTEGER)")
110         con.commit()
111         con.close()
112         with pytest.raises(dbmod.EncryptedDatabaseError) as ei:
113             with dbmod.connect_encrypted(p):
114                 pass
115         assert "muneem db encrypt" in str(ei.value) # actionable guidance, not a cryptic
error
116
117
118     # --- CLI: db rotate-key / db backup
-----
119
120
121     def _new_encrypted_ledger(path):
122         with dbmod.connect_encrypted(path) as conn:
123             dbmod.init_db(conn)
124             dbmod.insert_document(conn, source="file", sha256="s", status="parsed")
125
126
127     def test_db_rotate_key_cli(tmp_path):
128         led = tmp_path / "ledger.db"
129         _new_encrypted_ledger(led)
130         old = db_key.peek_key()
131         res = runner.invoke(app, ["db", "rotate-key", "--db", str(led)])
132         assert res.exit_code == 0, res.output
133         new = db_key.peek_key()
134         assert new and new != old
135         with dbmod.connect(led, key=new) as conn: # opens under the new key
136             assert dbmod.get_document_by_sha(conn, "s") is not None
137         with pytest.raises(dbmod.EncryptedDatabaseError): # old key no longer works
138             with dbmod.connect(led, key=old):
139                 pass
140
141
142     def test_db_backup_cli(tmp_path):
143         led = tmp_path / "ledger.db"
144         _new_encrypted_ledger(led)
145         dest = tmp_path / "led.backup.db"
146         res = runner.invoke(app, ["db", "backup", str(dest), "--db", str(led)])
147         assert res.exit_code == 0, res.output
148         assert dest.exists() and not dbmod.is_plaintext_sqlite(dest)
149         with dbmod.connect(dest, key=db_key.peek_key()) as conn:
150             assert dbmod.get_document_by_sha(conn, "s") is not None
151
152
153     def test_db_rotate_key_refuses_when_no_encrypted_ledger(tmp_path):
154         res = runner.invoke(app, ["db", "rotate-key", "--db", str(tmp_path / "nope.db")])
155         assert res.exit_code == 1 and "db encrypt" in res.output
156

```

24. user (2026-06-10T16:17:04.619Z)

```

1     """The hand-rolled migration runner: baseline apply, idempotency, legacy back-fill,
forward
2     migrations, baseline-drift detection, and running inside an encrypted connection.
3     """

```

```

4
5     import pytest
6
7     from muneem import db as dbmod
8     from muneem import migrations
9     from muneem.errors import MigrationError
10
11     _BASE = dbmod._SCHEMA
12     _VER = dbmod.SCHEMA_VERSION # 4
13
14
15     def _applied(conn):
16         return {r[0]: r[1] for r in conn.execute("SELECT version, name FROM
schema_migrations")}
17
18
19     def test_fresh_db_applies_baseline():
20         with dbmod.connect(":memory:") as conn:
21             # Isolate baseline behaviour from the real forward migrations (migrations=[]).
22             v = migrations.run_migrations(
23                 conn, baseline_sql=_BASE, baseline_version=_VER, migrations=[]
24             )
25             assert v == _VER
26             assert conn.execute("PRAGMA user_version").fetchone()[0] == _VER
27             assert _VER in _applied(conn) and _applied(conn)[_VER] == "baseline"
28             names = {r[0] for r in conn.execute("SELECT name FROM sqlite_master WHERE
type='table'")}
29             assert {"documents", "offers", "schema_migrations"} <= names
30
31
32     def test_default_path_applies_provenance_v5():
33         # init_db uses the real MIGRATIONS list ? baseline (4) + provenance (5).
34         with dbmod.connect(":memory:") as conn:
35             dbmod.init_db(conn)
36             assert conn.execute("PRAGMA user_version").fetchone()[0] == 5
37             applied = _applied(conn)
38             assert applied.get(5) == "provenance"
39             names = {r[0] for r in conn.execute("SELECT name FROM sqlite_master WHERE
type='table'")}
40             assert "provenance" in names
41
42
43     def test_idempotent_rerun_changes_nothing():
44         with dbmod.connect(":memory:") as conn:
45             migrations.run_migrations(conn, baseline_sql=_BASE, baseline_version=_VER)
46             before = _applied(conn)
47             migrations.run_migrations(conn, baseline_sql=_BASE, baseline_version=_VER)
48             assert _applied(conn) == before
49
50
51     def test_legacy_db_is_backfilled_without_re_running_ddl():
52         with dbmod.connect(":memory:") as conn:
53             # Pre-framework init: schema present + user_version set, but no
schema_migrations table.
54             conn.executescript(_BASE)
55             conn.execute(f"PRAGMA user_version = {_VER}")
56             conn.execute("INSERT INTO documents (source, sha256) VALUES ('file', 'abc')")
57             migrations.run_migrations(conn, baseline_sql=_BASE, baseline_version=_VER)
58             assert _VER in _applied(conn) # baseline recorded
59             assert conn.execute("SELECT count(*) FROM documents").fetchone()[0] == 1 # data
survived
60
61
62     def test_forward_migration_applies_once_and_bumps_version():
63         m = migrations.Migration(

```

```

64         version=_VER + 1, name="add_demo_col", sql="ALTER TABLE documents ADD COLUMN
demo TEXT"
65     )
66     with dbmod.connect(":memory:") as conn:
67         v = migrations.run_migrations(
68             conn, baseline_sql=_BASE, baseline_version=_VER, migrations=[m]
69         )
70         assert v == _VER + 1
71         assert conn.execute("PRAGMA user_version").fetchone()[0] == _VER + 1
72         assert "demo" in {r[1] for r in conn.execute("PRAGMA table_info(documents)")}
73         # Re-run must NOT re-ALTER (that would raise "duplicate column").
74         v2 = migrations.run_migrations(
75             conn, baseline_sql=_BASE, baseline_version=_VER, migrations=[m]
76         )
77         assert v2 == _VER + 1
78
79
80     def test_baseline_drift_is_detected():
81         with dbmod.connect(":memory:") as conn:
82             migrations.run_migrations(conn, baseline_sql=_BASE, baseline_version=_VER)
83             with pytest.raises(MigrationError): # baseline edited without a migration
84                 migrations.run_migrations(
85                     conn, baseline_sql=_BASE + "\n-- edited\n", baseline_version=_VER
86                 )
87
88
89     def test_runs_inside_an_encrypted_connection(tmp_path):
90         pytest.importorskip("sqlcipher3")
91         key = "00112233445566778899aabbccddeeff00112233445566778899aabbccddeeff"
92         with dbmod.connect(tmp_path / "led.db", key=key) as conn:
93             dbmod.init_db(conn) # the production path through the runner, encrypted
94             assert _VER in {r[0] for r in conn.execute("SELECT version FROM
schema_migrations")}
95

```

25. user (2026-06-10T16:17:05.041Z)

```

1     """The passphrase-wrapped recovery file ? round-trip, wrong-passphrase / corruption
rejection,
2     no-secret-leak, and the CLI setup-recovery ? (lost keychain) ? recover flow. Fabricated
keys only.
3     """
4
5     import pytest
6     from typer.testing import CliRunner
7
8     from muneem import recovery
9     from muneem.recovery import RecoveryError
10
11     runner = CliRunner()
12
13     _DBKEY = "00112233445566778899aabbccddeeff00112233445566778899aabbccddeeff"
14     _PASS = "correct horse battery staple"
15
16
17     def test_wrap_unwrap_roundtrip():
18         payload = recovery.wrap_key(_DBKEY, _PASS)
19         assert recovery.unwrap_key(payload, _PASS) == _DBKEY
20
21
22     def test_wrong_passphrase_is_rejected():
23         payload = recovery.wrap_key(_DBKEY, _PASS)
24         with pytest.raises(RecoveryError):
25             recovery.unwrap_key(payload, "a different passphrase")
26

```

```

27
28 def test_tampered_ciphertext_is_rejected():
29     payload = recovery.wrap_key(_DBKEY, _PASS)
30     payload["ciphertext"] = payload["ciphertext"][:-4] + "AAAA" # flip the tail
31     with pytest.raises(RecoveryError):
32         recovery.unwrap_key(payload, _PASS)
33
34
35 def test_short_passphrase_refused():
36     with pytest.raises(RecoveryError):
37         recovery.wrap_key(_DBKEY, "short")
38
39
40 def test_recovery_file_leaks_neither_key_nor_passphrase(tmp_path):
41     p = tmp_path / "muneem-recovery.json"
42     recovery.write_recovery_file(p, _DBKEY, _PASS)
43     blob = p.read_text(encoding="utf-8")
44     assert _DBKEY not in blob and _PASS not in blob # only the *wrapped* key is on disk
45     assert recovery.read_recovery_file(p, _PASS) == _DBKEY # but it round-trips
46
47
48 def test_read_missing_or_garbage_file_raises(tmp_path):
49     with pytest.raises(RecoveryError):
50         recovery.read_recovery_file(tmp_path / "nope.json", _PASS)
51     bad = tmp_path / "bad.json"
52     bad.write_text("not json", encoding="utf-8")
53     with pytest.raises(RecoveryError):
54         recovery.read_recovery_file(bad, _PASS)
55
56
57 # --- CLI: setup-recovery -> lose keychain -> recover
58 -----
59
60 def test_cli_setup_recovery_then_recover(tmp_path, monkeypatch, _isolate_keychain):
61     pytest.importorskip("sqlcipher3")
62     from muneem import db as dbmod
63     from muneem import db_key
64     from muneem.cli import app
65
66     led = tmp_path / "ledger.db"
67     with dbmod.connect_encrypted(led) as conn:
68         dbmod.init_db(conn)
69         dbmod.insert_document(conn, source="file", sha256="s", status="parsed")
70     original = db_key.peek_key()
71     assert original
72
73     rec = tmp_path / "muneem-recovery.json"
74     monkeypatch.setattr("getpass.getpass", lambda prompt="": _PASS)
75     r1 = runner.invoke(app, ["db", "setup-recovery", "--out", str(rec)])
76     assert r1.exit_code == 0, r1.output
77     assert rec.exists()
78
79     _isolate_keychain.clear() # simulate a lost keychain
80     assert db_key.peek_key() is None
81
82     r2 = runner.invoke(app, ["db", "recover", str(rec), "--db", str(led)])
83     assert r2.exit_code == 0, r2.output
84     assert db_key.peek_key() == original # key restored to the keychain
85     with dbmod.connect(led, key=db_key.peek_key()) as conn:
86         assert dbmod.get_document_by_sha(conn, "s") is not None
87
88
89 def test_cli_setup_recovery_passphrase_mismatch(tmp_path, monkeypatch,
_isolate_keychain):

```

```

90     from muneem import db as dbmod
91     from muneem.cli import app
92
93     with dbmod.connect_encrypted(tmp_path / "ledger.db") as conn:
94         dbmod.init_db(conn)
95     answers = iter(["passphrase-one", "passphrase-two"])
96     monkeypatch.setattr("getpass.getpass", lambda prompt="": next(answers))
97     r = runner.invoke(app, ["db", "setup-recovery", "--out", str(tmp_path / "r.json")])
98     assert r.exit_code == 1 and "did not match" in r.output
99
100
101 def test_cli_recover_wrong_passphrase(tmp_path, monkeypatch, _isolate_keychain):
102     from muneem.cli import app
103
104     rec = tmp_path / "r.json"
105     recovery.write_recovery_file(rec, _DBKEY, _PASS)
106     monkeypatch.setattr("getpass.getpass", lambda prompt="": "the wrong passphrase
entirely")
107     r = runner.invoke(app, ["db", "recover", str(rec), "--db", str(tmp_path /
"absent.db")])
108     assert r.exit_code == 1 and ("wrong passphrase" in r.output or "corrupt" in
r.output)
109

```

26. user (2026-06-10T16:17:10.624Z)

```

1     """The OS-keychain wrappers: graceful read, attributed (value-free) write, swallowed
delete.
2
3     The autouse confstest fixture swaps ``muneem.secrets`` for an in-memory store, so we
capture the
4     *real* wrapper functions at import time and drive them by patching the underlying
``keyring`` lib.
5     """
6
7     import keyring
8     import pytest
9     from keyring.errors import KeyringError
10
11     import muneem.secrets as secretsmod
12     from muneem.errors import KeychainError
13
14     # Captured before the autouse keychain fixture rebinds the module attributes per-test.
15     _real_get = secretsmod.get_secret
16     _real_set = secretsmod.set_secret
17     _real_delete = secretsmod.delete_secret
18
19
20 def test_get_secret_returns_none_when_keyring_unavailable(monkeypatch):
21     def boom(service, name):
22         raise KeyringError("no backend")
23
24     monkeypatch.setattr(keyring, "get_password", boom)
25     assert _real_get("anything") is None # graceful: absence, not a crash
26
27
28 def test_get_secret_returns_the_value(monkeypatch):
29     monkeypatch.setattr(keyring, "get_password", lambda s, n: "v" if n == "k" else None)
30     assert _real_get("k") == "v"
31     assert _real_get("missing") is None
32
33
34 def test_set_secret_raises_keychain_error_with_name_but_not_value(monkeypatch):
35     def boom(service, name, value):
36         raise KeyringError("locked")
37

```

```

37
38     monkeypatch.setattr(keyring, "set_password", boom)
39     with pytest.raises(KeychainError) as ei:
40         _real_set("db-key", "super-secret-value")
41     msg = str(ei.value)
42     assert "db-key" in msg # the name is safe to surface
43     assert "super-secret-value" not in msg # the value must NEVER appear
44
45
46 def test_delete_secret_swallows_keyring_errors(monkeypatch):
47     def boom(service, name):
48         raise KeyringError("nope")
49
50     monkeypatch.setattr(keyring, "delete_password", boom)
51     _real_delete("k") # must not raise
52

```

27. user (2026-06-10T16:17:11.758Z)

```

1     import json
2     import os
3     from pathlib import Path
4
5     import pytest
6
7
8     @pytest.fixture(autouse=True)
9     def _isolate_keychain(monkeypatch):
10         """Never touch the real OS keychain in tests.
11
12         Replaces :mod:`muneem.secrets` with an in-memory store so the suite cannot pollute
the
13         developer's keychain (e.g. minting a DB key) and runs cleanly in headless CI where
no
14         keyring backend exists. Tests that need specific keychain behaviour re-patch on top.
15         """
16         store: dict[str, str] = {}
17
18         def _set(name, value):
19             store[name] = value
20
21         monkeypatch.setattr("muneem.secrets.get_secret", store.get)
22         monkeypatch.setattr("muneem.secrets.set_secret", _set)
23         monkeypatch.setattr("muneem.secrets.delete_secret", lambda name: store.pop(name,
None))
24         return store
25
26
27     @pytest.fixture
28     def snapshot():
29         def _assert_snapshot(name: str, data: list | dict):
30             snapshot_dir = Path("testdata/snapshots")
31             snapshot_dir.mkdir(parents=True, exist_ok=True)
32             snapshot_file = snapshot_dir / f"{name}.json"
33
34             if os.environ.get("UPDATE_SNAPSHOTS") == "1":
35                 with snapshot_file.open("w", encoding="utf-8") as f:
36                     json.dump(data, f, indent=2, ensure_ascii=False)
37                 return True
38
39             if not snapshot_file.exists():
40                 pytest.fail(f"Snapshot not found at {snapshot_file}. Run with
UPDATE_SNAPSHOTS=1.")
41
42             with snapshot_file.open("r", encoding="utf-8") as f:

```

```

43         expected = json.load(f)
44
45         assert data == expected, f"Data does not match snapshot {name}"
46         return True
47
48     return _assert_snapshot
49

```

28. user (2026-06-10T16:17:11.791Z)

```

1     """Tests for the ingest-path password resolver (muneem.statement_unlock).
2
3     Synthetic encrypted PDFs (pypdf), no real statements. The OS keychain is replaced with
an in-memory
4     dict so tests never touch the real keyring (and run cleanly in headless CI). No real
password.
5     """
6
7     import json
8     from pathlib import Path
9
10    import pypdf
11    import pytest
12
13    from muneem import statement_unlock
14
15
16    def _pdf(path: Path, password: str | None = None) -> Path:
17        writer = pypdf.PdfWriter()
18        writer.add_blank_page(width=200, height=200)
19        if password:
20            writer.encrypt(password)
21        with open(path, "wb") as fh:
22            writer.write(fh)
23        return path
24
25
26    @pytest.fixture(autouse=True)
27    def fake_keychain(monkeypatch):
28        """Replace muneem.secrets with an in-memory store (no real keyring, CI-safe)."""
29        store: dict[str, str] = {}
30        monkeypatch.setattr(statement_unlock.secrets, "get_secret", store.get)
31        monkeypatch.setattr(
32            statement_unlock.secrets, "set_secret", lambda k, v: store.__setitem__(k, v)
33        )
34        # default: no configured candidates unless a test overrides
35        monkeypatch.setattr(statement_unlock, "candidate_passwords", list)
36        return store
37
38
39    def test_plaintext_pdf_needs_no_password(tmp_path):
40        assert statement_unlock.resolve_password(_pdf(tmp_path / "plain.pdf")) == ""
41
42
43    def test_resolves_via_configured_candidate(tmp_path, monkeypatch):
44        monkeypatch.setattr(statement_unlock, "candidate_passwords", lambda: ["wrong",
"secret123"])
45        pdf = _pdf(tmp_path / "enc.pdf", password="secret123")
46        assert statement_unlock.resolve_password(pdf, interactive=False) == "secret123"
47
48
49    def test_no_prompt_failure_returns_none(tmp_path, monkeypatch):
50        monkeypatch.setattr(statement_unlock, "candidate_passwords", lambda: ["wrong"])
51        pdf = _pdf(tmp_path / "enc.pdf", password="secret123")
52        assert statement_unlock.resolve_password(pdf, interactive=False) is None

```



```

53
54
55     def test_prompt_resolves_then_caches_for_next_time(tmp_path, monkeypatch,
fake_keychain):
56         pdf = _pdf(tmp_path / "enc.pdf", password="secret123")
57         monkeypatch.setattr(statement_unlock.getpass, "getpass", lambda prompt="":
"secret123")
58         assert statement_unlock.resolve_password(pdf, interactive=True) == "secret123"
59         # the winning password is cached in the (fake) keychain
60         assert "secret123" in json.loads(fake_keychain[statement_unlock._CACHE_KEY])
61
62         # a *new* file with the same password now unlocks from the cache ? no prompt
63         def _no_prompt(prompt=""):
64             raise AssertionError("should not prompt once the password is cached")
65
66         monkeypatch.setattr(statement_unlock.getpass, "getpass", _no_prompt)
67         pdf2 = _pdf(tmp_path / "enc2.pdf", password="secret123")
68         assert statement_unlock.resolve_password(pdf2, interactive=True) == "secret123"
69
70
71     def test_resolves_via_cached_candidate_without_prompting(tmp_path, monkeypatch,
fake_keychain):
72         statement_unlock._remember("cachedpw")
73         pdf = _pdf(tmp_path / "enc.pdf", password="cachedpw")
74
75         def _no_prompt(prompt=""):
76             raise AssertionError("cached candidate should resolve before any prompt")
77
78         monkeypatch.setattr(statement_unlock.getpass, "getpass", _no_prompt)
79         assert statement_unlock.resolve_password(pdf, interactive=True) == "cachedpw"
80
81
82     def test_remember_caps_and_orders_most_recent_first(fake_keychain):
83         for i in range(25):
84             statement_unlock._remember(f"pw{i}")
85         cached = statement_unlock._cached_candidates()
86         assert len(cached) == statement_unlock._MAX_CACHED # bounded
87         assert cached[0] == "pw24" # most-recently-used first
88         assert "pw0" not in cached # oldest aged out
89
90
91     def test_remember_ignores_empty_and_whitespace(fake_keychain):
92         statement_unlock._remember("")
93         statement_unlock._remember(" ")
94         assert statement_unlock._cached_candidates() == []
95
96
97     def test_remember_moves_existing_to_front_without_duplicating(fake_keychain):
98         statement_unlock._remember("a")
99         statement_unlock._remember("b")
100         statement_unlock._remember("a") # re-seen -> MRU front, no duplicate
101         assert statement_unlock._cached_candidates() == ["a", "b"]
102
103
104     def test_password_never_appears_in_logs(tmp_path, monkeypatch, fake_keychain, caplog):
105         import logging
106
107         pdf = _pdf(tmp_path / "enc.pdf", password="topsecretpw")
108         monkeypatch.setattr(statement_unlock.getpass, "getpass", lambda prompt="":
"topsecretpw")
109         with caplog.at_level(logging.DEBUG):
110             assert statement_unlock.resolve_password(pdf, interactive=True) == "topsecretpw"
111             assert "topsecretpw" not in caplog.text # never logged on the success path
112

```

29. user (2026-06-10T16:20:06.313Z)

```
439: # Keychain-first with rollback: if the file rekey fails, the keychain still matches the
file.
440: db_key.set_key(fresh)
442: dbmod.rekey_database(db_path, current, fresh)
444: db_key.set_key(current) # roll back; the file is still under the old key
531: db_key.set_key(key)
```

30. user (2026-06-10T16:20:07.136Z)

```
17:from muneem.errors import EncryptedDatabaseError # re-exported so db.EncryptedDatabaseError
works
146:     raise EncryptedDatabaseError(
154: except sqlalchemy3.DatabaseError as exc:
156:     raise EncryptedDatabaseError(
203:     raise EncryptedDatabaseError(
216: copied. Raises :class:`EncryptedDatabaseError` if ``db_path`` is not a plaintext SQLite
file.
222:     raise EncryptedDatabaseError(
250:         raise EncryptedDatabaseError(
270: :class:`EncryptedDatabaseError` if ``current_key`` is wrong or the file is not an
encrypted
288: it stays encrypted at rest. Raises :class:`EncryptedDatabaseError` if ``key`` is wrong.
```

31. user (2026-06-10T16:22:00.041Z)

Structured output provided successfully

32. assistant (2026-06-10T16:22:07.972Z)

Review complete ? findings delivered via structured output: 1 critical (rotate-key key-lifecycle atomicity), 2 high (keychain read-failure overwriting the only DB key; `db backup` able to destroy the live ledger), 6 medium, and 5 low findings across the crypto seam, migration runner, CLI, and redaction layers.